[ADS5034-01] Computer Vision

14. Image Generative Model & Wrap Up

Jae-Young Choi

Sungkyunkwan University

Outline



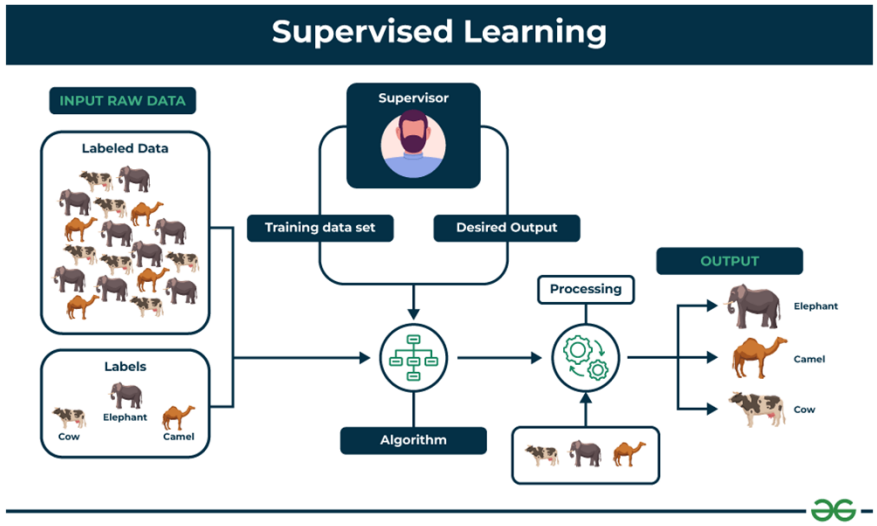
- Generative Model
  - Overview
  - PixelCNN
  - Variational Autoencoder (VAE)
  - Generative Adversarial Networks (GAN)
  - Diffusion Model
- Wrap up

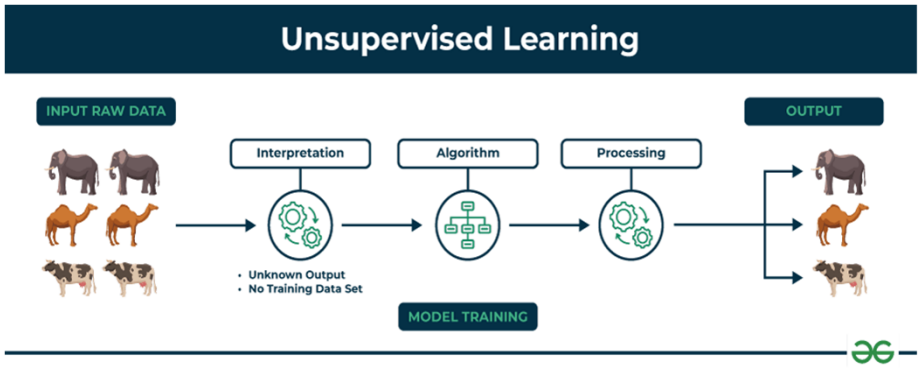
Generative Model

- Overview -

Supervised vs Unsupervised Learning

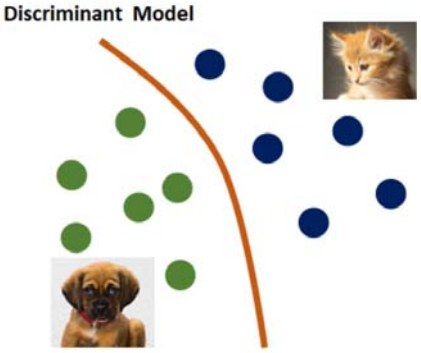
- The main difference between the two is the type of data used to train the computer
  - **Supervised learning** involves a human “teacher” or “supervisor.” Their role is to **feed the computer with labeled data or examples** consisting of a combination of problems and solutions.
  - **Unsupervised learning** is used when there is no labeled data or instructions for the computer to follow. Instead, the computer tries to identify the **underlying structure or patterns** in the data without any assistance.

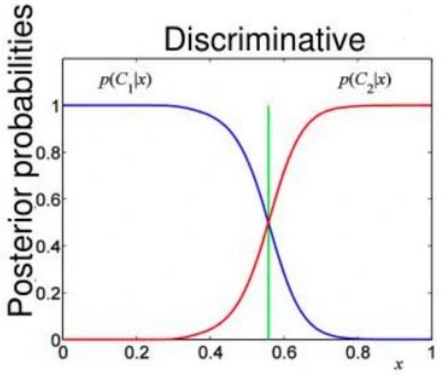




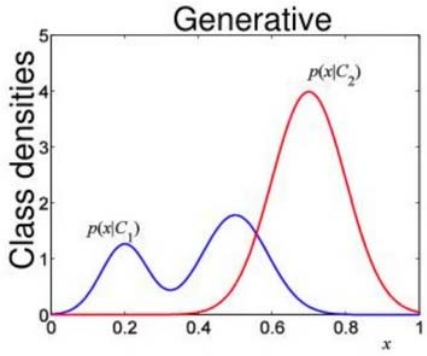
Discriminative model vs. Generative model

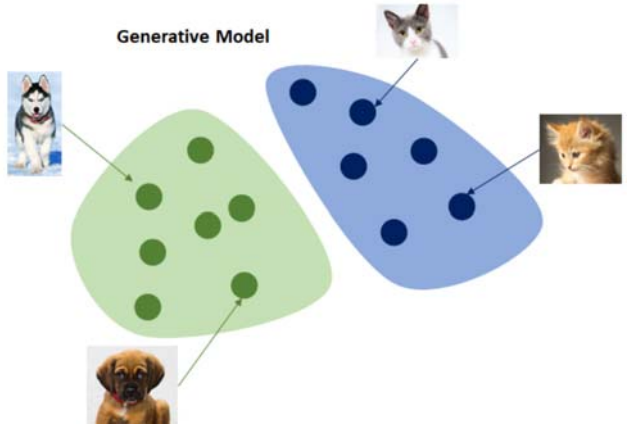
- The majority of **discriminative models** are used for supervised machine learning. They separating the data points into different classes and learning the boundaries using probability estimates and maximum likelihood.
- As the name suggests, **generative models** can be used to generate new data points. These models are usually used in unsupervised machine learning problems.





vs.

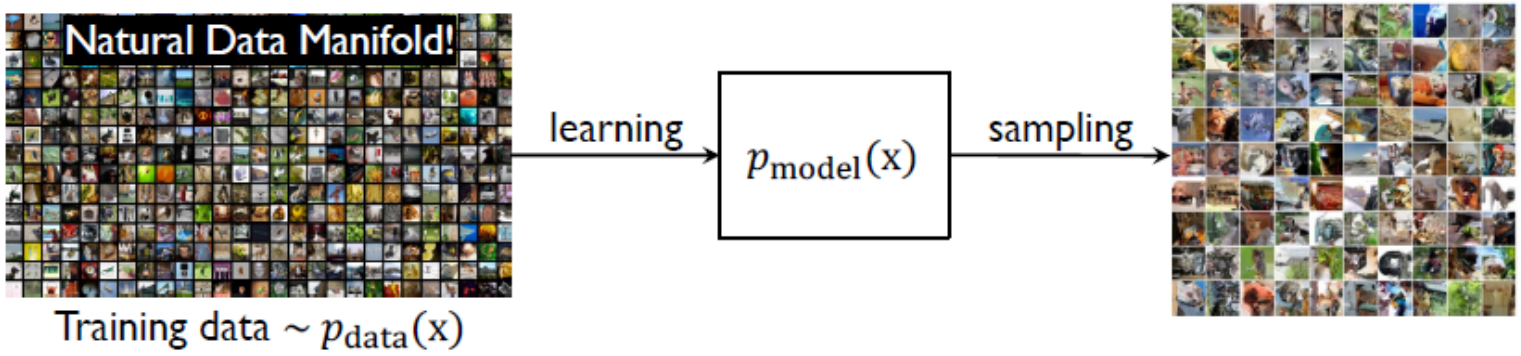




Generative Modeling

- Given training data, generate new samples from same distribution

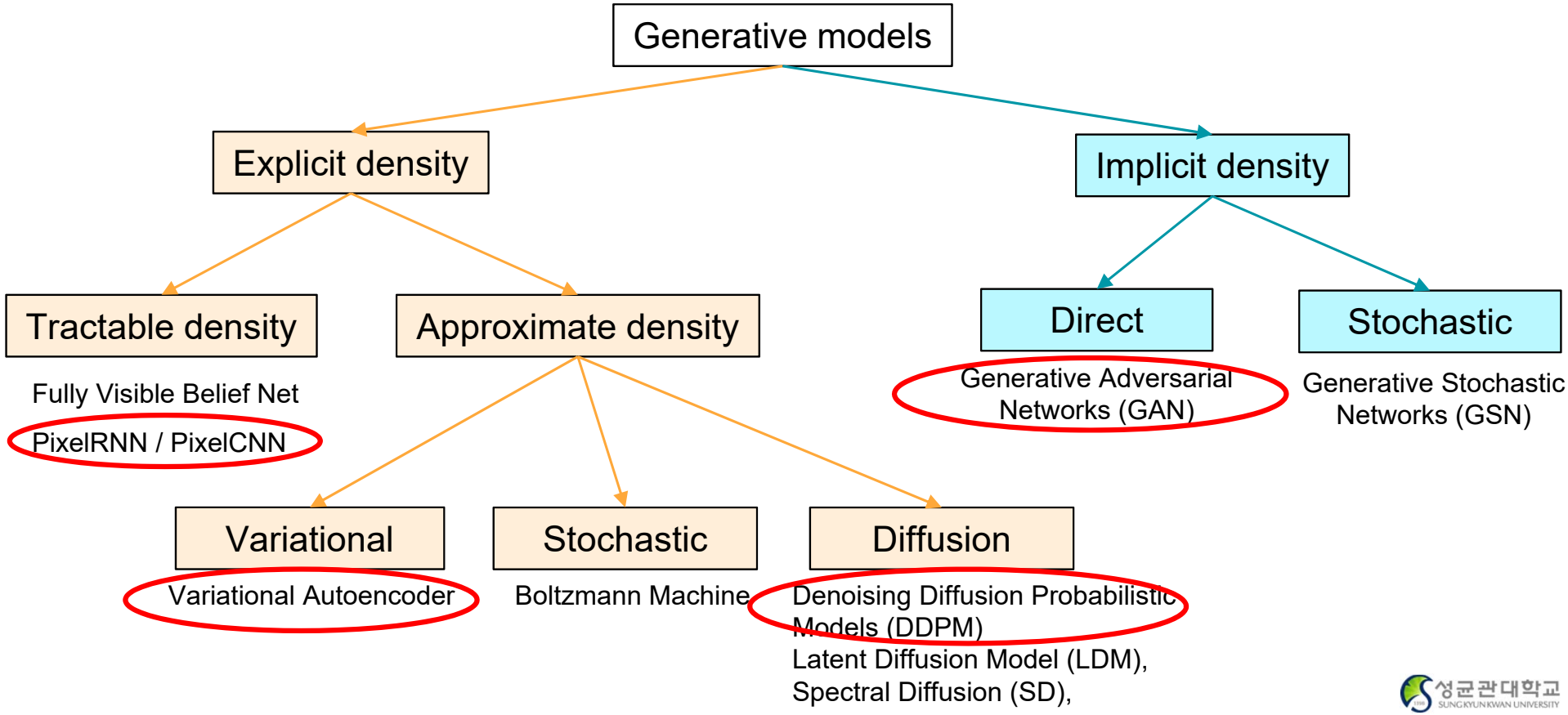
Objectives:
1. Learn $p_{model}(x)$ that approximates $p_{data}(x)$
2. Sampling new x from $p_{model}(x)$



- Formulate as density estimation problems:
  - Explicit density estimation: explicitly define and solve for $p_{model}(x)$
  - Implicit density estimation: learn model that can sample from $p_{model}(x)$ without explicitly defining it.

Generative Modeling

- Taxonomy of Generative Models



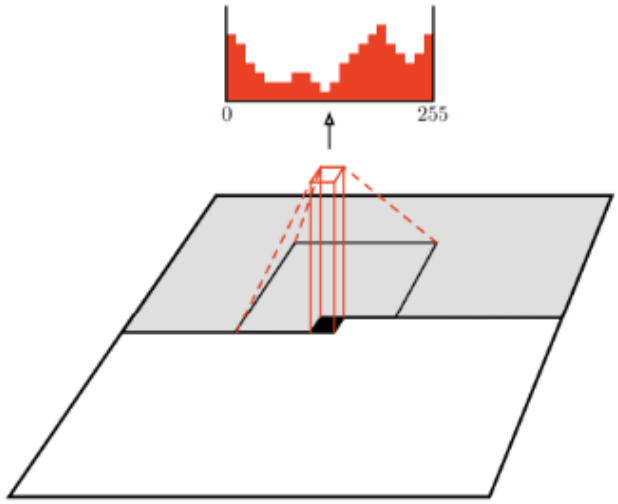
PixelCNN

Pixel Recurrent Neural Networks, CVPR 2016 (https://arxiv.org/abs/1601.06759)

Conditional Image Generation with PixelCNN Decoders, CVPR 2016 (https://arxiv.org/abs/1606.05328)

PixelCNN

- Main purpose of PixelCNN model is to model (apology for tautology) pixel distribution $p(x)$, where x=(x_0,..,x_n) vector if pixel values of given image.
- It employs **autoregressive models** to do that: they just use plain **chain rule** for joint distribution: $p(\mathbf{x}) = p(x_0) \prod_1^n p(x_i|x_{i<})$

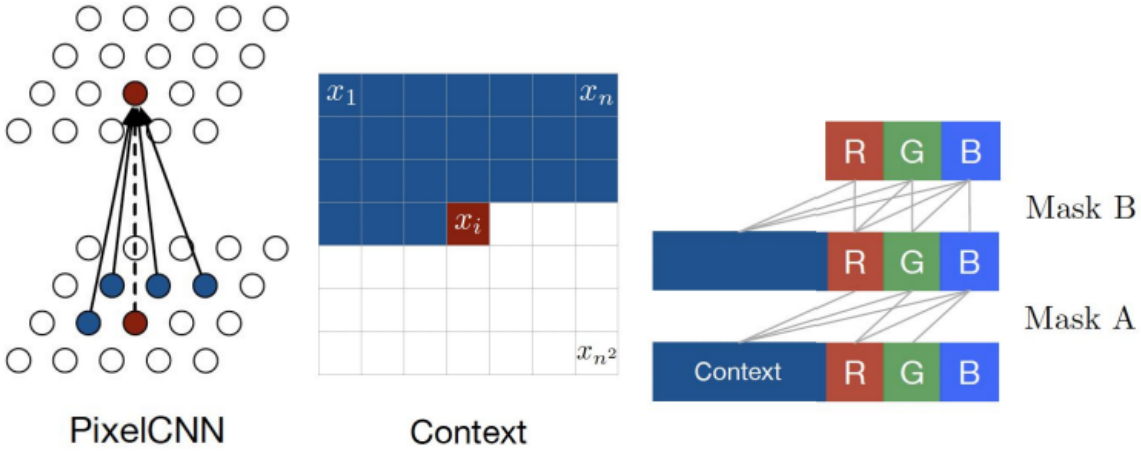


1	1	1	1	1
1	1	1	1	1
1	1	0	0	0
0	0	0	0	0
0	0	0	0	0

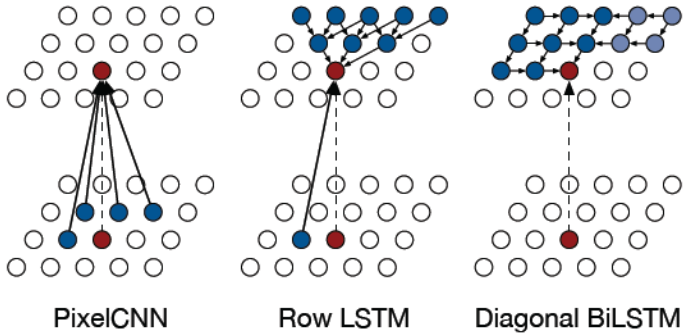
PixelCNN

- Instead of RNNs, this model consists of stacked convolutional layers
- Generates the target pixel using previously generated nearby pixels
- PixelCNNs are much faster to train than PixelRNNs because convolutions are inherently easier to parallelize

$$p(x_{i,R}|x_{<i}) \rightarrow p(x_{i,G}|x_{<i}, x_{i,R}) \rightarrow p(x_{i,B}|x_{<i}, x_{i,R}, x_{i,G})$$



<Visualization of the input-to-state and state-to-state mappings>



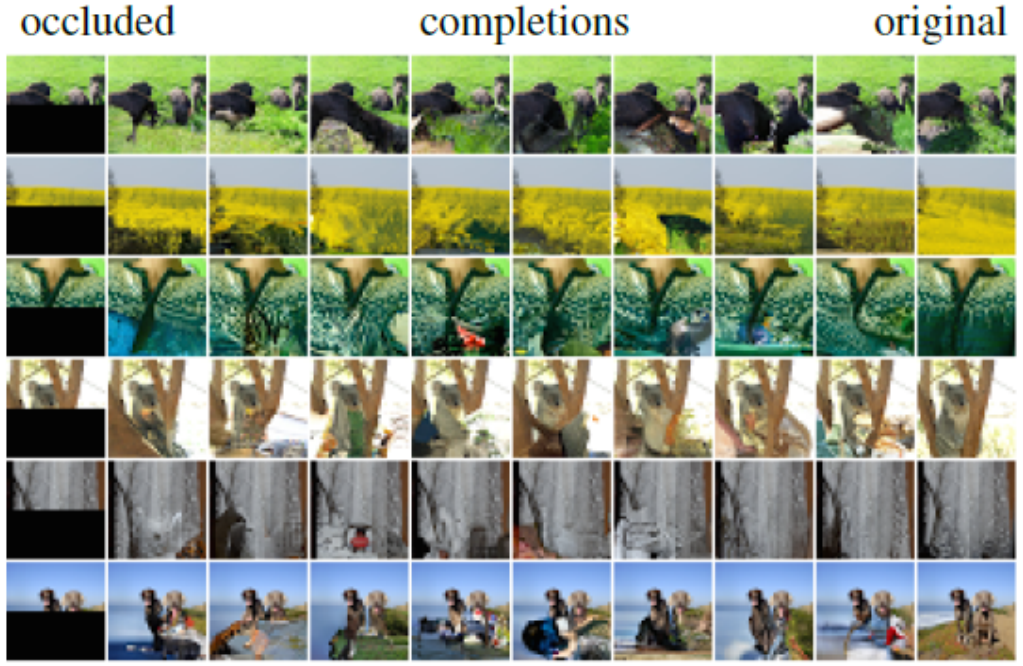
Mask B: relaxed from mask A by allowing the connection from the same channel to generate now. Used at the 2nd+ layer

Mask A: rely on previously generated pixels(context) + previously generated channels on the target pixel. Used at the very first layer

PixelCNN

- Pros
  - Can explicitly compute likelihood p(x)
  - Easy to optimize
- Con
  - Slow due to sequential generation

PixelCNN	Row LSTM	Diagonal BiLSTM
7×7 conv mask A		
Multiple residual blocks: (see fig 5)		
Conv 3×3 mask B	Row LSTM i-s: 3×1 mask B s-s: 3×1 no mask	Diagonal BiLSTM i-s: 1×1 mask B s-s: 1×2 no mask
ReLU followed by 1×1 conv, mask B (2 layers)		
256-way Softmax for each RGB color (Natural images) or Sigmoid (MNIST)		



<Image completions sampled from a model
that was trained on 32x32 ImageNet images>

Variational AutoEncoder (VAE)

Auto-Encoding Variational Bayes, ICRL 2014 (https://arxiv.org/abs/1312.6114)

AutoEncoder

- PixelRNN/CNNs define tractable density function, optimize likelihood of training data:

$$p_\theta(x) = \prod_{i=1}^{n} p_\theta(x_i | x_1, ..., x_{i-1})$$

- Variational Autoencoders (VAEs) define intractable density function with latent ***z***:

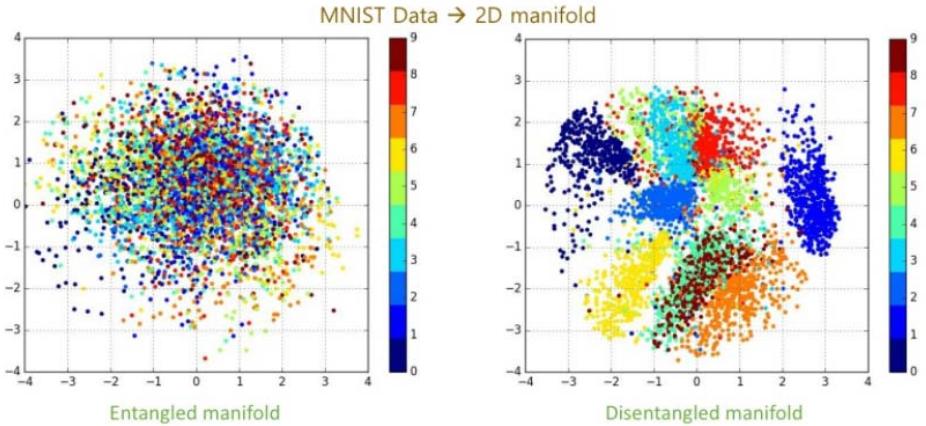
$$p_\theta(x) = \int p_\theta(z) p_\theta(x|z) dz$$

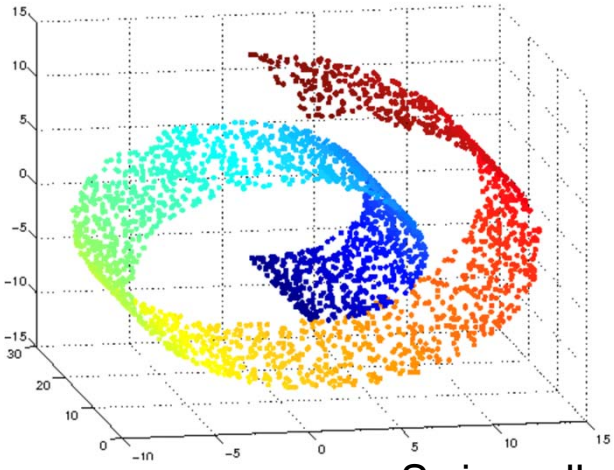
- No dependencies among pixels, can generate all pixels at the same time
- Cannot optimize directly, derive and optimize lower bound on likelihood instead

Why latent *z*?

AutoEncoder

- Discovering most important features




Swiss roll

- Dimensionality Reduction (Taxonomy)

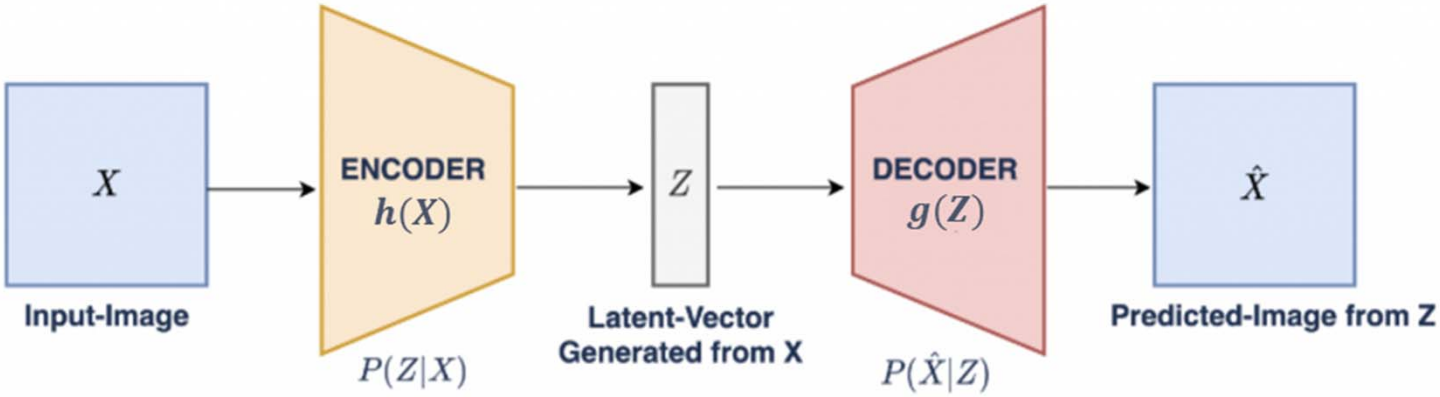
Dimensionality Reduction

Linear
- Principal Component Analysis (PCA)
- Linear Discriminant Analysis (LDA)
- etc..

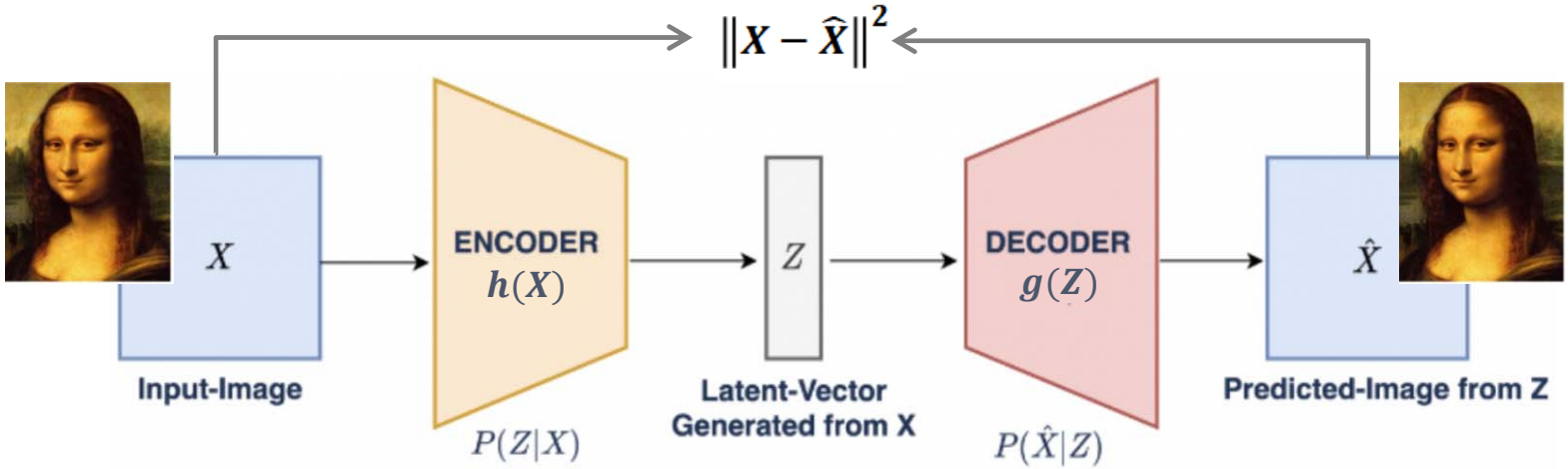
Non-Linear
- Autoencoders (AE)
- t-distributed stochastic neighbor embedding (t-SNE)
- Isomap
- Locally-linear embedding (LLE)
- etc..

AutoEncoder

- Unsupervised approach for learning a lower-dimensional feature representation from unlabeled training data
- Trained with **reconstruction loss** :
  - Features are trained to reconstruct original data (the input itself)
- Input can be anything (no assumption about the data)
- Z is usually smaller than X (dimensionality reduction), **Why?**
  - Want features to capture meaningful factors of variation in data



AutoEncoder



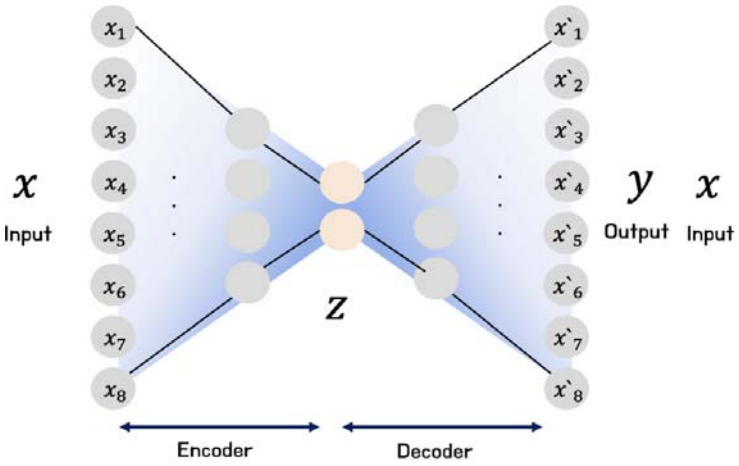
- Encoder
  - Embedding
  - Manifold learning
  - Feature extraction
- Encoder can be used to initialize a supervised model

- Decoder
  - It is no longer used after training
- L2 loss function (softmax, etc)

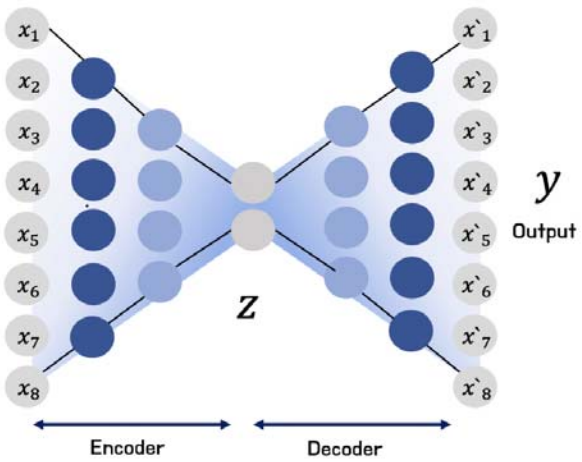
$$L = \sum_{X \in D} \|X - g(h(X))\|^2 = \|X - \hat{X}\|^2$$

AutoEncoder

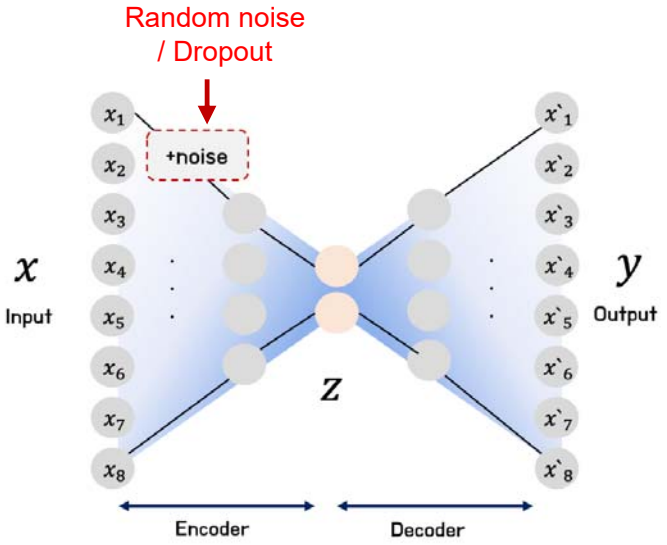
- Other Methods



Autoencoder



Stacked Autoencoder



Denoising Autoencoder

AutoEncoder

- Applications

Image Coloring



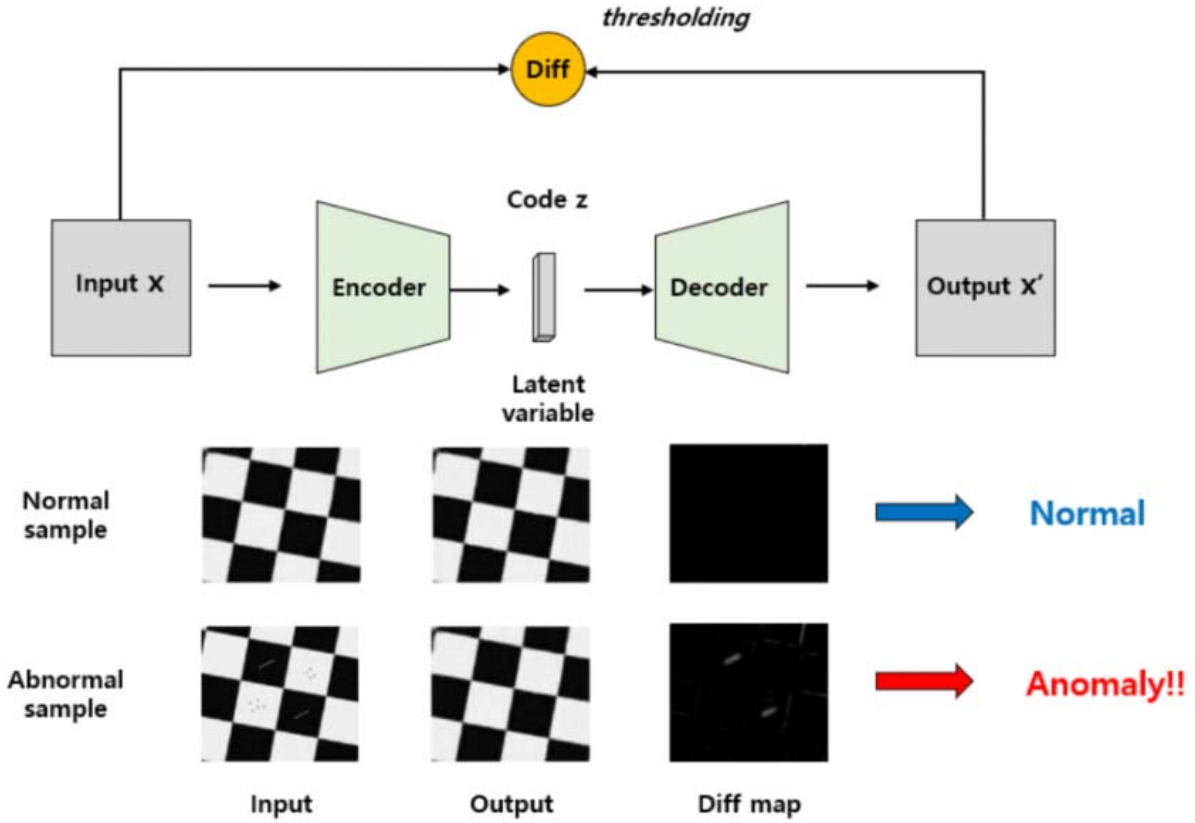
Before After

Image Noise Reduction



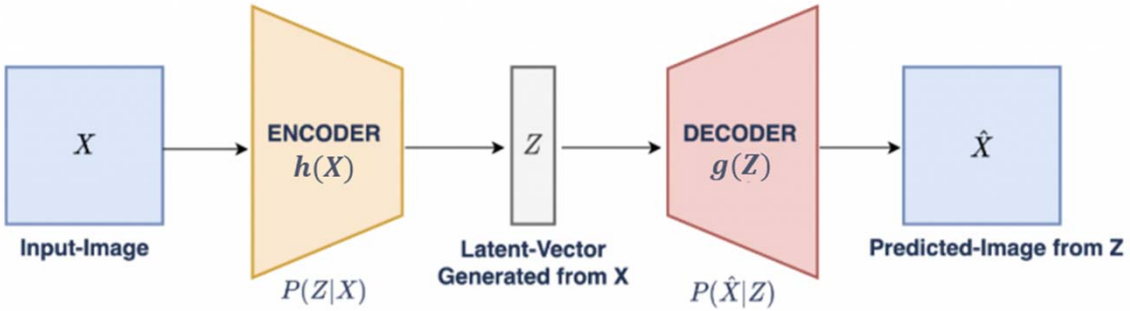
Before After

Anomaly Detection

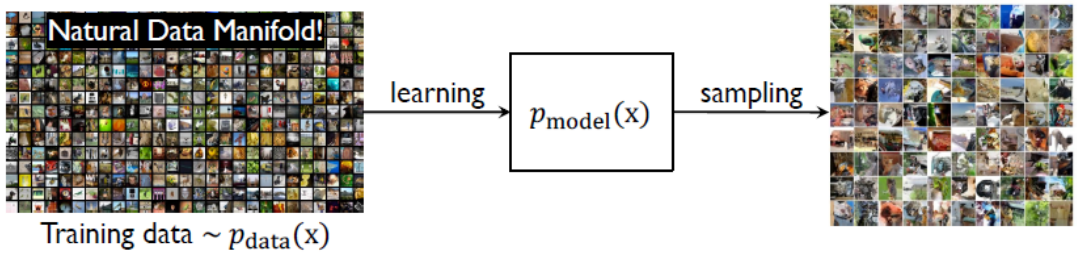


AutoEncoder

- Autoencoders can reconstruct data, and can learn features to initialize a supervised model
- Features capture factors of variation in training data
- But we can't generate new images because we don't know the space of ***Z***.



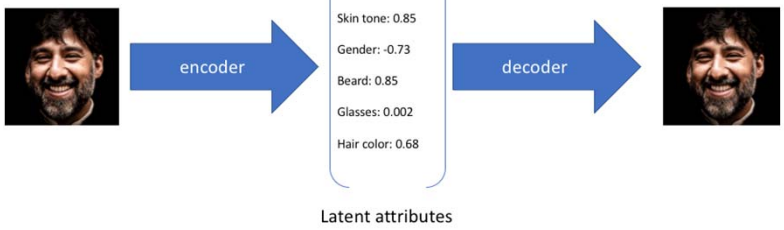
Can we make autoencoder a generative model?



Variational AutoEncoders

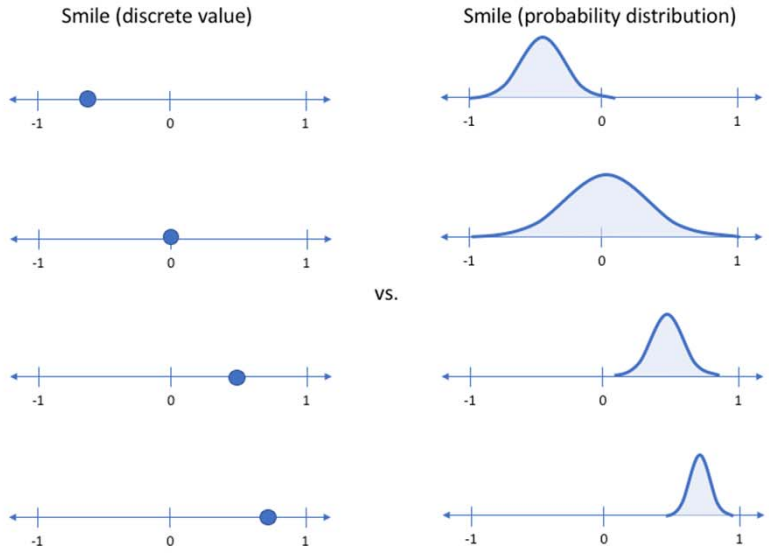
- **Probabilistic spin on autoencoders** - will let us sample from the model to generate data!
- Assume training data is generated from the distribution of unobserved (latent) representation z

Intuition (remember from autoencoders): x is an image, z is latent factors used to generate x: attributes, orientation, etc.



https://www.jeremyjordan.me/variational-autoencoders/





Variational AutoEncoders

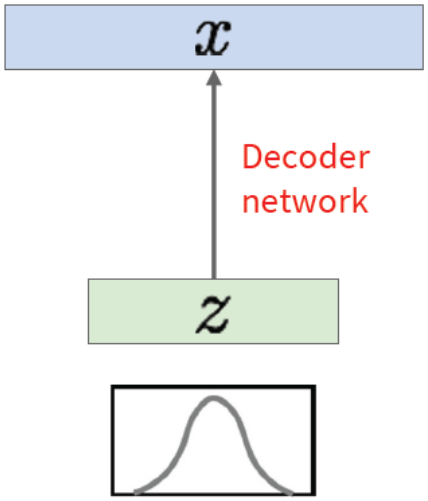
- Using a variational autoencoder, we can describe latent attributes in probabilistic terms.
- We want to estimate the true parameters θ^* of this generative model given training data x.

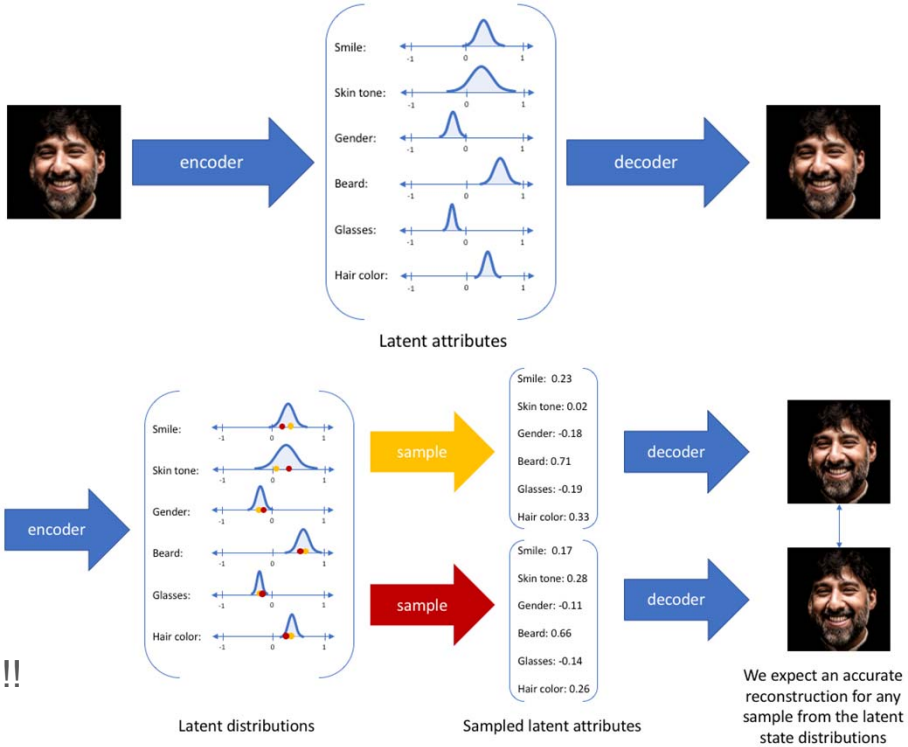
Sample from true conditional

$p_{\theta^*}(x \mid z^{(i)})$

Sample from true prior

$z^{(i)} \sim p_{\theta^*}(z)$





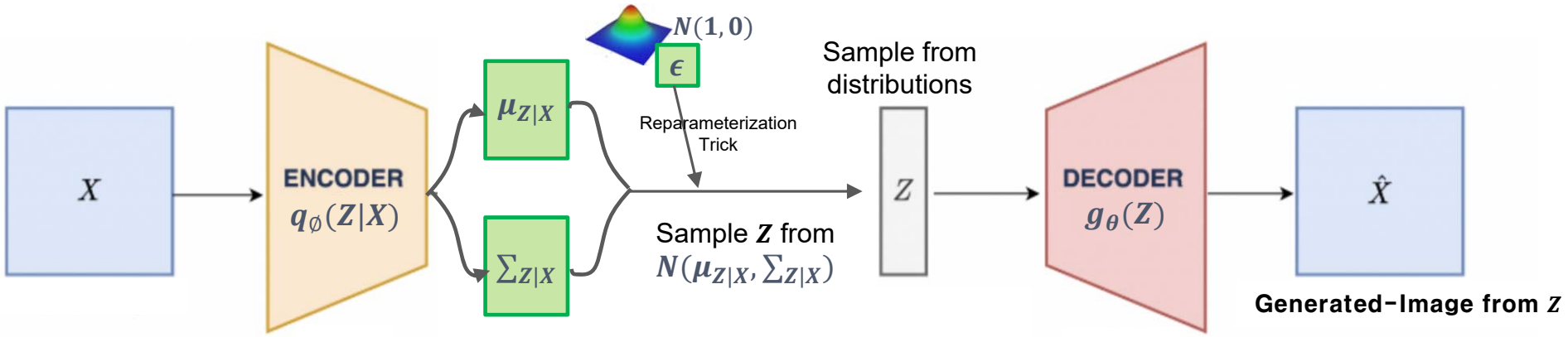
Learn model parameters to maximize likelihood of training data!!

$$p_\theta(x) = \int p_\theta(z) p_\theta(x|z) dz$$

https://www.jeremyjordan.me/variational-autoencoders/

Variational AutoEncoders

- Intractable to compute $p(X|Z)$ for every Z **Data likelihood:** $p_\theta(x) = \int p_\theta(z)p_\theta(x|z)dz$
- Solution: In addition to modeling $p_\theta(X|Z)$, learn $q_\theta(Z|X)$ that approximates the true posterior $p_\theta(Z|X)$. That is, $Z \sim p(Z|\mathrm{X}) \approx q_\emptyset(Z|X)$
- Will see that the approximate posterior allows us to derive a lower bound on the data likelihood that is tractable, which we can optimize.
- Variational inference is to approximate the unknown posterior distribution from only the observed data X



$$\underbrace{\mathbf{E}_z\left[\log p_\theta(x^{(i)} \mid z)\right] - D_{KL}(q_\phi(z \mid x^{(i)}) \,||\, p_\theta(z))}_{\mathcal{L}(x^{(i)}, \theta, \phi)}$$

Variational AutoEncoders

- We want to maximize the data Likelihood

$$\log p_\theta(x^{(i)}) = \mathbf{E}_{z\sim q_\phi(z|x^{(i)})}\left[\log p_\theta(x^{(i)})\right] \quad (p_\theta(x^{(i)}) \text{ Does not depend on } z)$$

$$= \mathbf{E}_z\left[\log \frac{p_\theta(x^{(i)} \mid z)p_\theta(z)}{p_\theta(z \mid x^{(i)})}\right] \quad \text{(Bayes' Rule)}$$

$$= \mathbf{E}_z\left[\log \frac{p_\theta(x^{(i)} \mid z)p_\theta(z)}{p_\theta(z \mid x^{(i)})}\frac{q_\phi(z \mid x^{(i)})}{q_\phi(z \mid x^{(i)})}\right] \quad \text{(Multiply by constant)}$$

$$= \mathbf{E}_z\left[\log p_\theta(x^{(i)} \mid z)\right] - \mathbf{E}_z\left[\log \frac{q_\phi(z \mid x^{(i)})}{p_\theta(z)}\right] + \mathbf{E}_z\left[\log \frac{q_\phi(z \mid x^{(i)})}{p_\theta(z \mid x^{(i)})}\right] \quad \text{(Logarithms)}$$

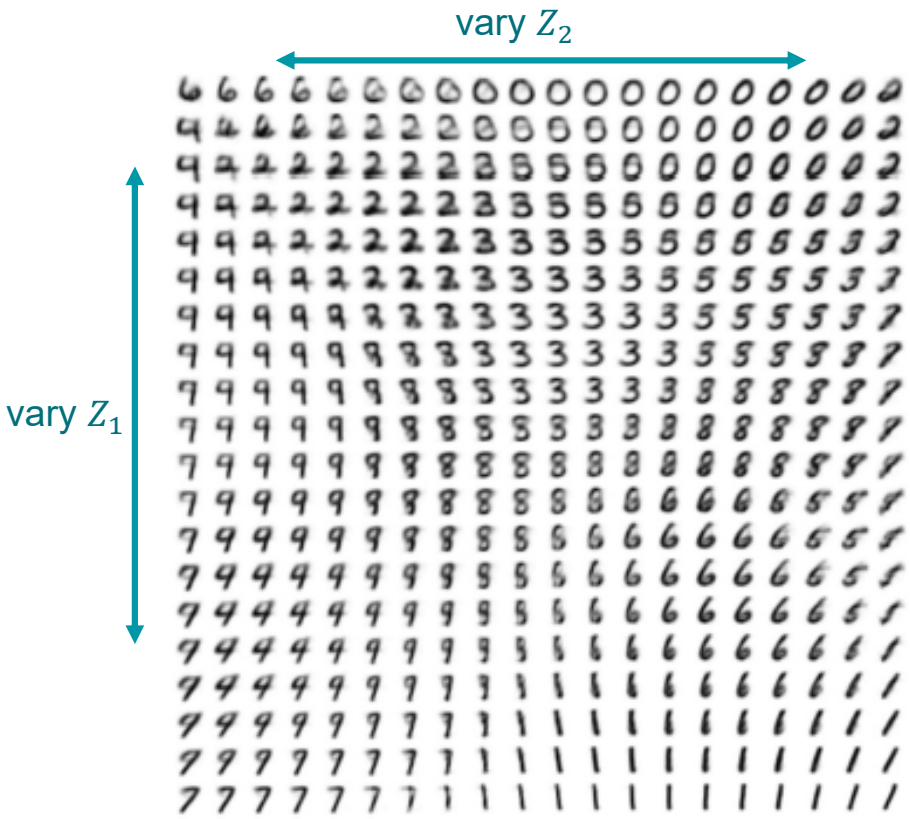
$$= \underbrace{\mathbf{E}_z\left[\log p_\theta(x^{(i)} \mid z)\right] - D_{KL}(q_\phi(z \mid x^{(i)}) \,||\, p_\theta(z))}_{\mathcal{L}(x^{(i)}, \theta, \phi)} + \underbrace{D_{KL}(q_\phi(z \mid x^{(i)}) \,||\, p_\theta(z \mid x^{(i)}))}_{\geq 0}$$

Reconstruction error　　Regularization error

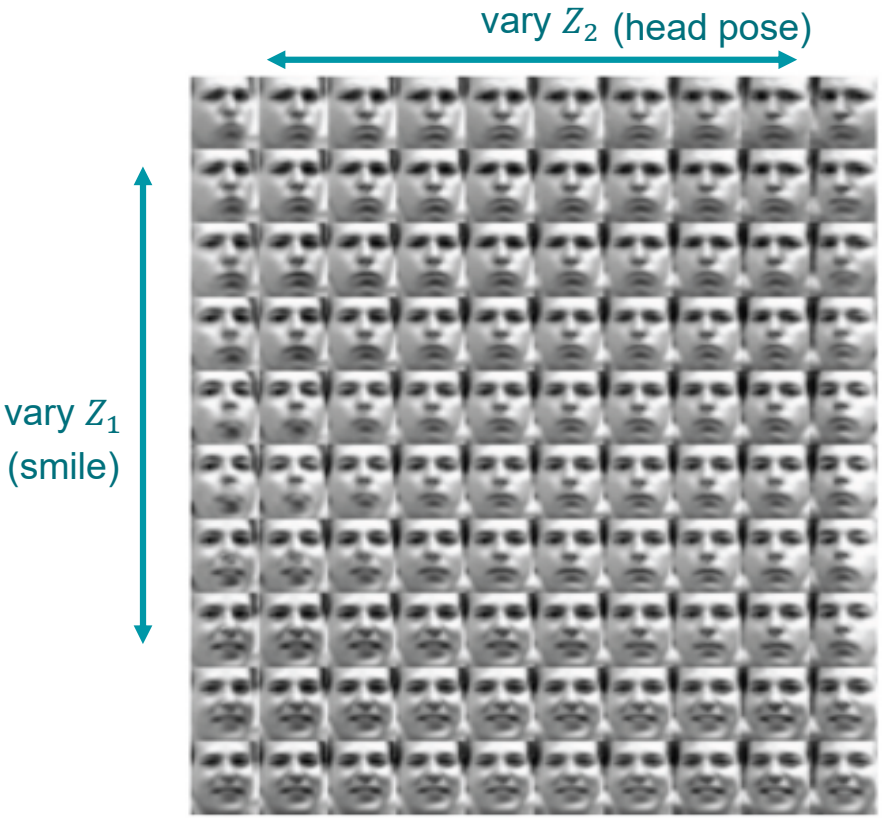
VAE ELBO : Evidence lower Bound

Variational AutoEncoders

- **Learned manifold :** Interpretable factors of variation



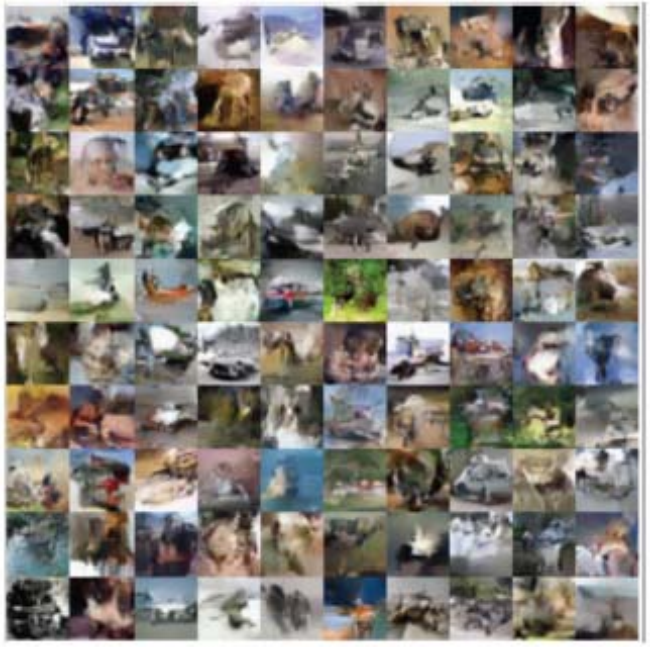
MNIST data



Facial expression

Variational AutoEncoders

- Generating Data



32x32 CIFAR-10



Labeled Faces in the Wild

Variational AutoEncoders

- Principled approach to generative models
  - Probabilistic spin to traditional autoencoders ⇒ allows generating data
  - Defines an intractable density ⇒ derive and optimize a (variational) lower bound
- Pros:
  - Interpretable latent space.
  - Allows inference of q(z|x), can be useful feature representation for other tasks
- Cons:
  - Maximizes lower bound of likelihood: working with a lower bound of likelihood (approximated)
  - Not as good evaluation as PixelRNN/PixelCNN (which uses actual likelihood)
  - Samples blurrier and lower quality compared to state-of-the-art (GANs)

Generative adversarial networks (GANs)

Tutorial : Generative Adversarial Networks, NeurIPS 2014 (https://arxiv.org/abs/1701.00160)

GANs

- PixelRNN/CNNs define tractable density function, optimize likelihood of training data

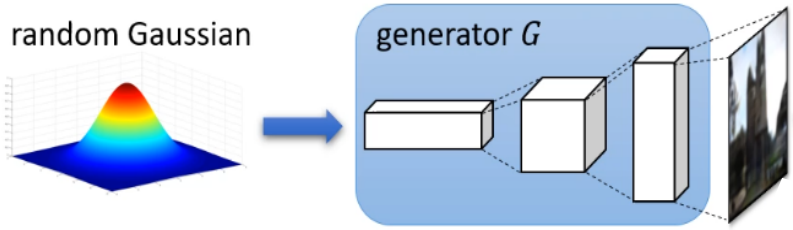
$$p_\theta(x) = \prod_{i=1}^{n} p_\theta(x_i | x_1, ..., x_{i-1})$$

- VAEs define intractable density function with latent z

$$p_\theta(x) = \int p_\theta(z) p_\theta(x|z) dz$$

  Cannot optimize directly, derive and optimize lower bound on likelihood instead

- What if we give up on explicitly modeling density, and just want ability to sample?
  - Solution : Sample from a simple distribution we can easily sample from, e.g. random noise. Learn transformation to training distribution



GANs

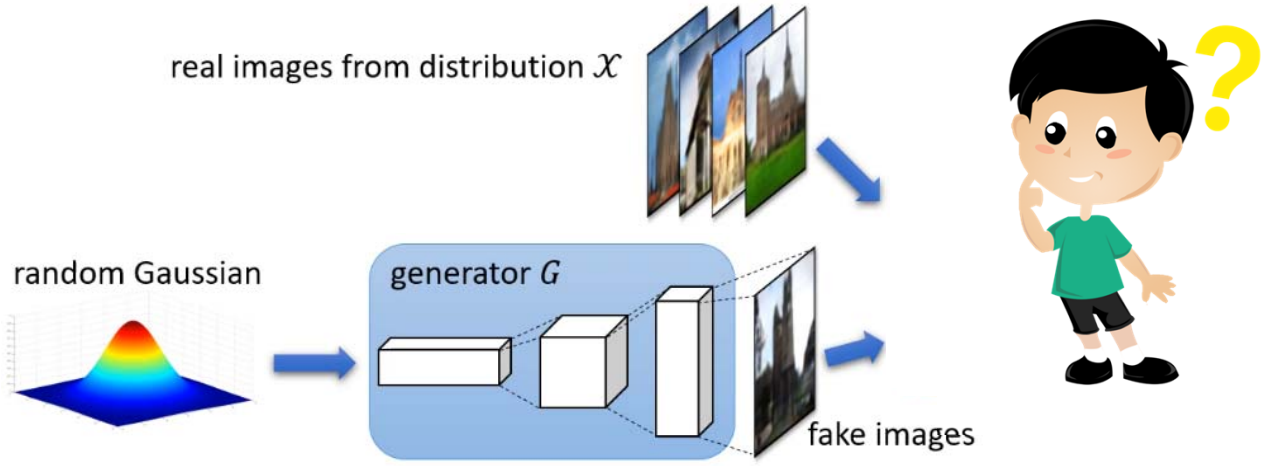
- But we don't know which sample z maps to which training image

  ⇒ can't learn by reconstructing training images

- A human inspecting all output?

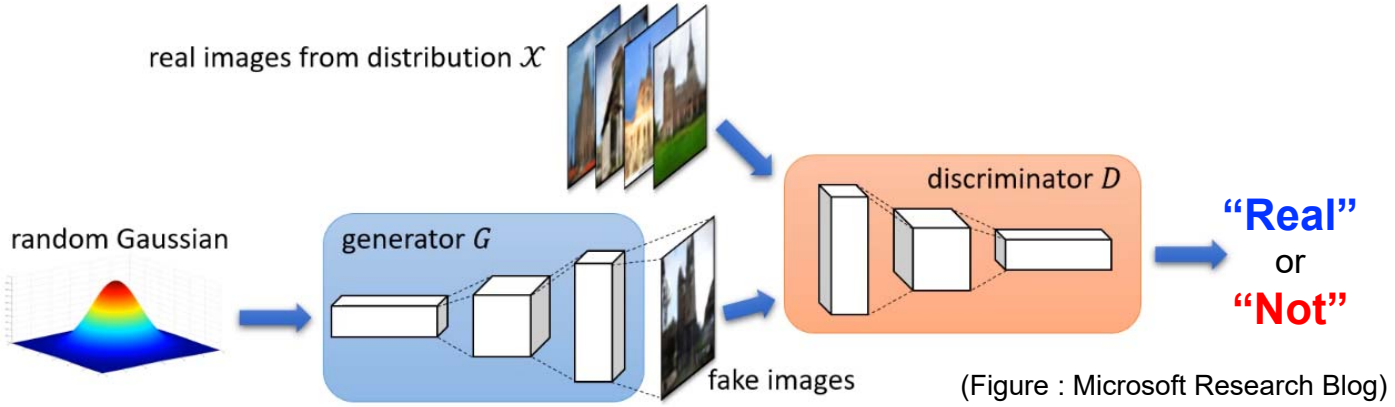
  ⇒ Solution : Use a discriminator network to tell whether the generate image is within data distribution ("real") or not

  ⇒ Training signal (Real/Fake) does not require human labeling (unsupervised)



GANs

- Two-player game
  - Discriminator network: try to distinguish between real and fake images
  - Generator network: try to pass the discriminator by generating real-looking images



(Figure : Microsoft Research Blog)

  - Train jointly in minimax game

Minimax objective function:

$$\min_{\theta_g} \max_{\theta_d} \left[\mathbb{E}_{x \sim p_{data}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z))) \right]$$

GANs

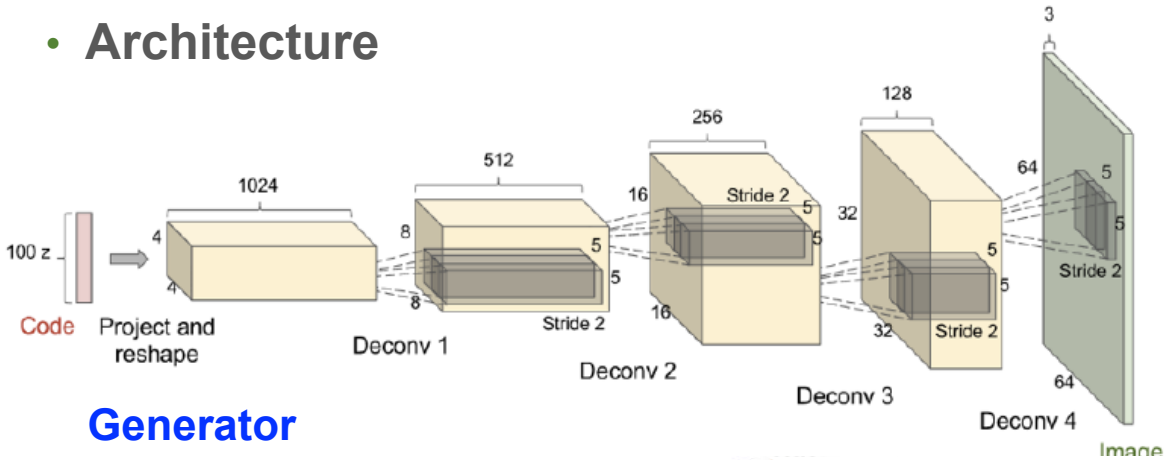
- **Input :** a real image in training data, or a fake image generated by Generator
- **Output :** probability that the input is a real image (with sigmoid)
- **Objective :** real vs. fake binary classification (cross-entropy)
- Alternatively solve:
  - Gradient Ascent by the Discriminator: $\max_{\theta_d} \left[\mathbb{E}_{x\sim p_{data}} \log D_{\theta_d}(x) + \mathbb{E}_{z\sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z)))\right]$
  - Gradient Descent by the Generator: $\min_{\theta_g} \mathbb{E}_{z\sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z)))$
- Putting all together, training is done jointly in minimax game
  - One is trying to maximize, while the other to minimize the objective function

$$\min_{\theta_g} \max_{\theta_d} \left[\mathbb{E}_{x\sim p_{data}} \log D_{\theta_d}(x) + \mathbb{E}_{z\sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z)))\right]$$

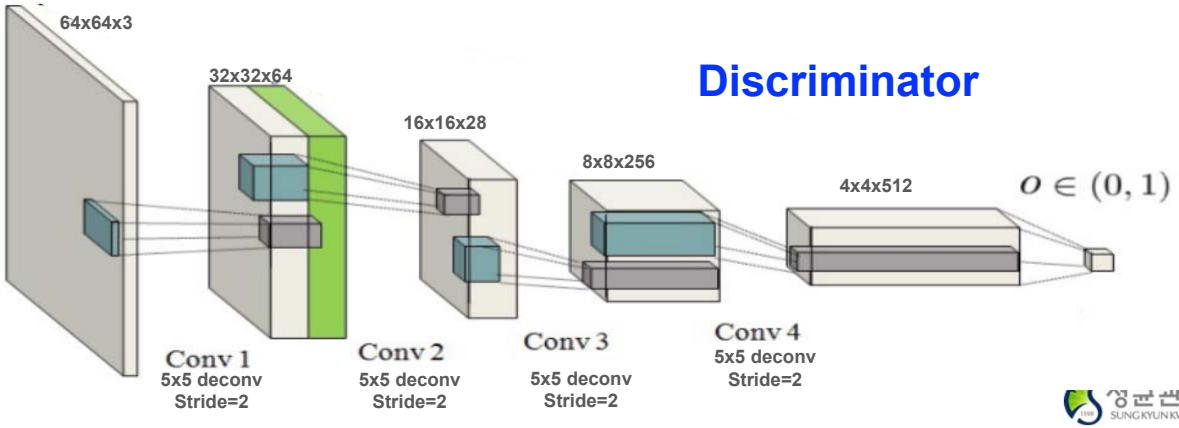
GANs

- **Deep Convolutional GAN (DCGAN)**
  - **Architecture**

UNSUPERVISED REPRESENTATION LEARNING WITH DEEP CONVOLUTIONAL GENERATIVE ADVERSARIAL NETWORKS ICLR 2016, (https://arxiv.org/pdf/1511.06434)



Generator



Discriminator

GANs

- **Generated samples**

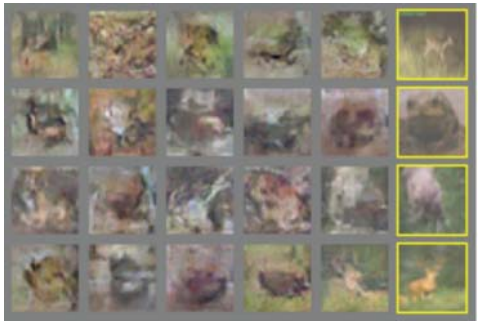




- **Tried deconv (G) and conv (D) combination for CIFAR10 experiment**
  - **The original GAN paper used fully-connected for most experiments**



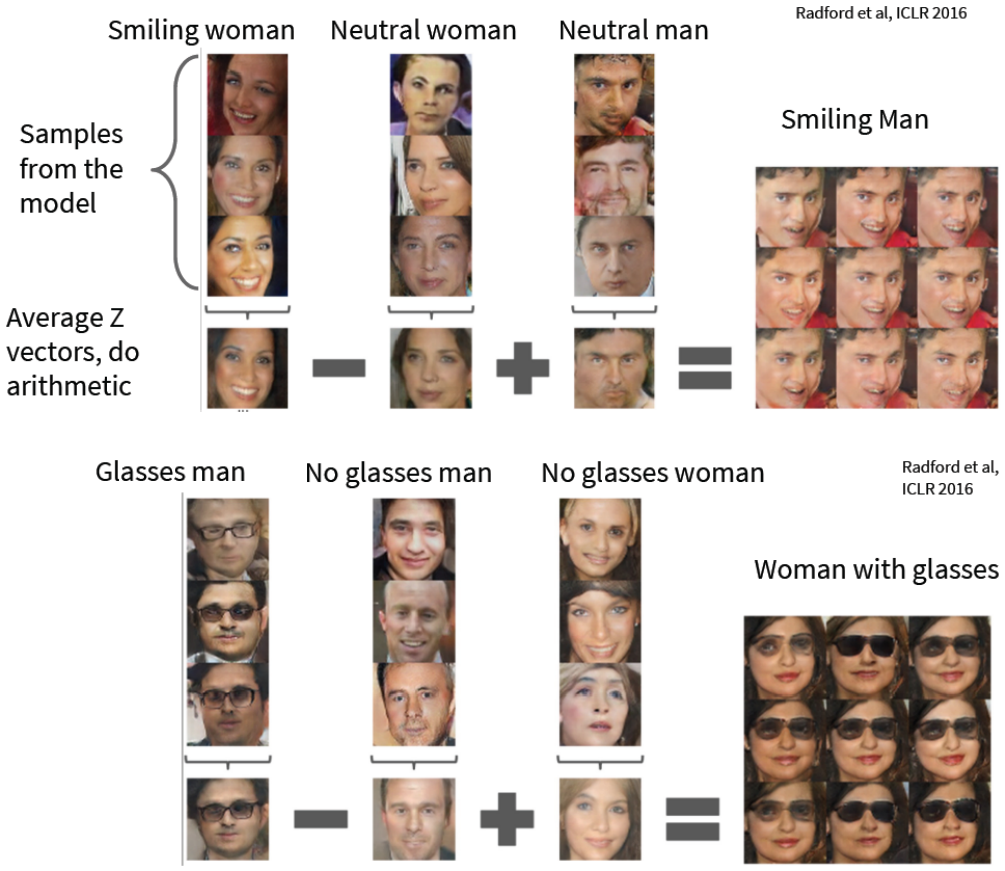
FC model trained on CIFAR10



Deconv-conv model trained on CIFAR10

GANs

- **Interpretable Vector Math**





A ”turn” vector was created from four averaged samples of faces looking left vs looking right by adding **interpolations** along this axis to random samples

GANs

- **The Evolution of GANs**
  - **Vanilla GAN:** This is the most basic form of GAN, where both the generator and discriminator are modeled using multi-layer perceptrons. The generator focuses on capturing data distribution, while the discriminator evaluates the probability that a given sample is real or synthetic.
  - **Conditional GAN (CGAN):** Here, both networks receive additional information such as class labels, making the model conditional. This allows the generation of data that is more specific to the given condition.
  - **Deep Convolutional GAN (DCGAN):** DCGANs employ CNN, making them more effective for tasks that involve image data. They are known for generating high-quality, high-resolution images.
  - **CycleGAN:** Used for image-to-image translation, CycleGAN can transform images from one domain to another without needing paired examples, such as turning summer landscapes into winter ones.
  - **Generative Adversarial Text to Image Synthesis:** These GANs can generate images from textual descriptions, bridging the gap between natural language and visual data.
  - **StyleGAN:** Known for generating high-quality and realistic images, StyleGAN allows for control over the style of the generated images, such as altering facial features in portraits.
  - **Super Resolution GAN (SRGAN):** SRGAN enhances the resolution of images, turning low-resolution inputs into high-resolution outputs without losing detail.

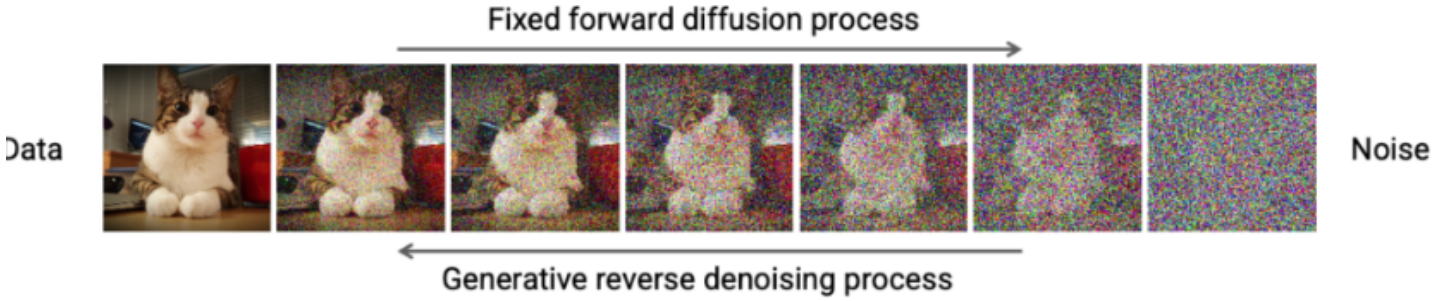
Diffusion Model

Denoising Diffusion Probabilistic Models, NeurIPS 2020 (https://arxiv.org/abs/2006.11239)

A Survey on Generative Diffusion Model, IEEE Transactions on Knowledge and Data Engineering 2022 (https://arxiv.org/abs/2209.02646)

Diffusion Model

- Diffusion Models are generative models that learn from the data during training and generate similar examples based on what they have learned
- In a nutshell, Diffusion Models work by corrupting training data through the addition of *Gaussian noise* (called ***Diffusion process***), and then learning how to recover the original information by reversing this noising process step by step (called ***Reverse process*** or ***Denoising process***).
- Once trained, these models can generate new data by sampling random Gaussian noise and passing it through the learned denoising process.

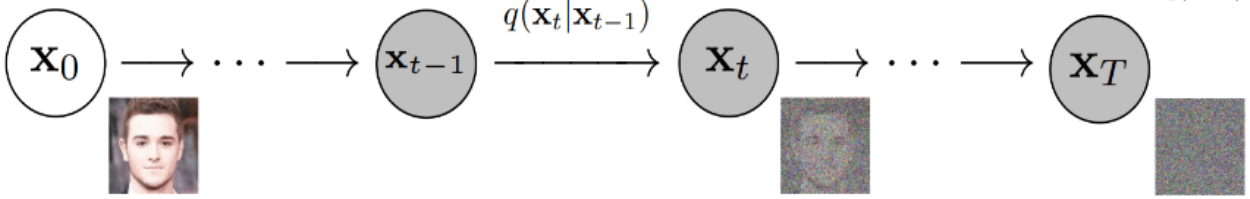


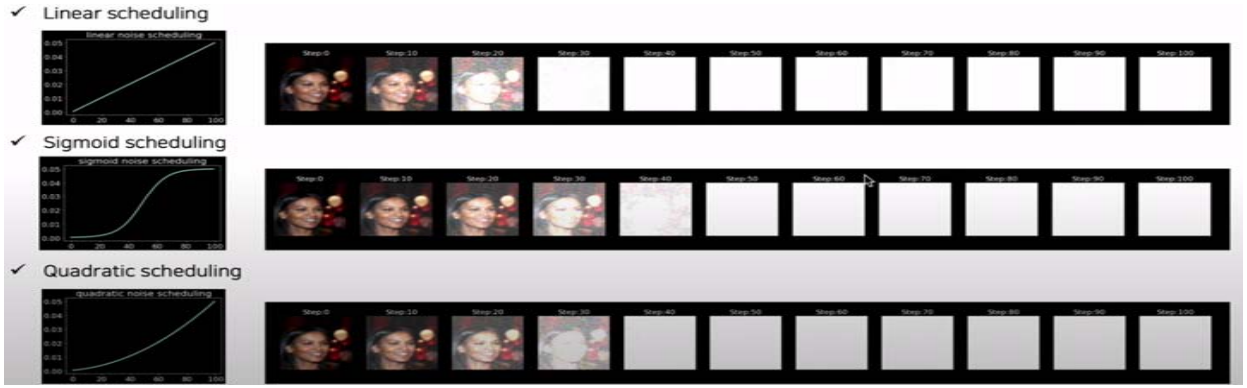
Diffusion Model

- **Forward Diffusion Process**
  - The forward diffusion process consists of gradually adding **Gaussian noise** to an input image step by step, for a total of *T* steps (Conditional Gaussian Distribution)
  - To formalize this process, we can view it as a fixed **Markov chain** with T steps, where the image at timestep t maps to its subsequent state at timestep t+1

$$q(\mathbf{x}_{1:T}|\mathbf{x}_0) := \prod_{t=1}^{T} q(\mathbf{x}_t|\mathbf{x}_{t-1}), \qquad q(\mathbf{x}_t|\mathbf{x}_{t-1}) := \mathcal{N}(\mathbf{x}_t; \sqrt{1-\beta_t}\mathbf{x}_{t-1}, \beta_t\mathbf{I})$$

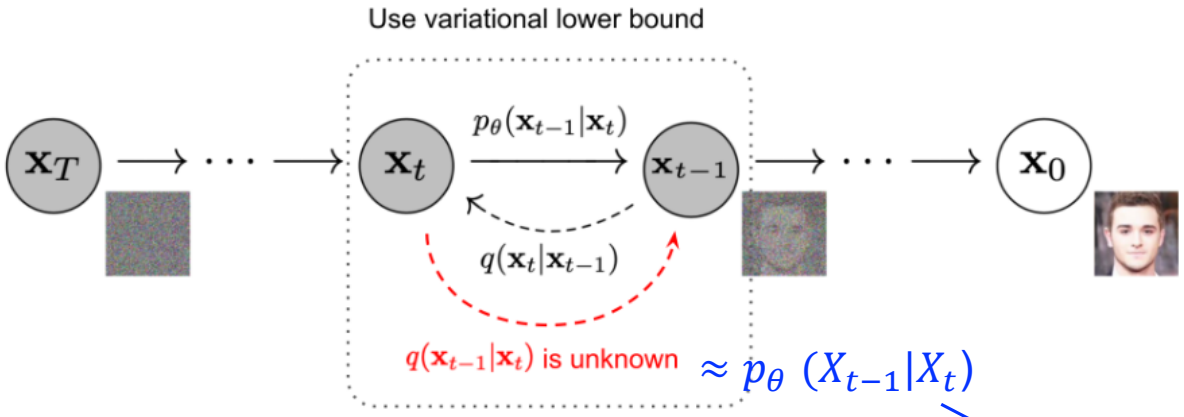
$$q(\mathbf{x}_t|\mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t}\mathbf{x}_0, (1-\bar{\alpha}_t)\mathbf{I})$$



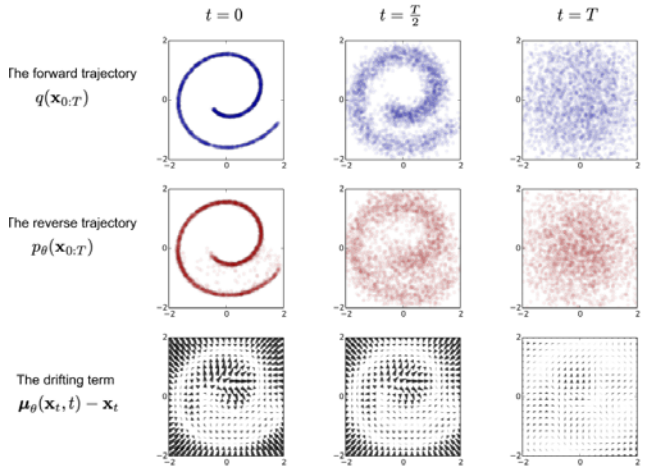


Diffusion Model

- **Reverse Diffusion Process**
  - The reverse diffusion process poses a computational challenge as the formulation of $q(x_{t-1}|x_t)$ becomes intractable or uncomputable
  - Generally speaking, the role of this neural network is to predict the whole noise present in the image at a certain timestep t. Then, by comparing this prediction with the real noise added to the image, the network can be trained.



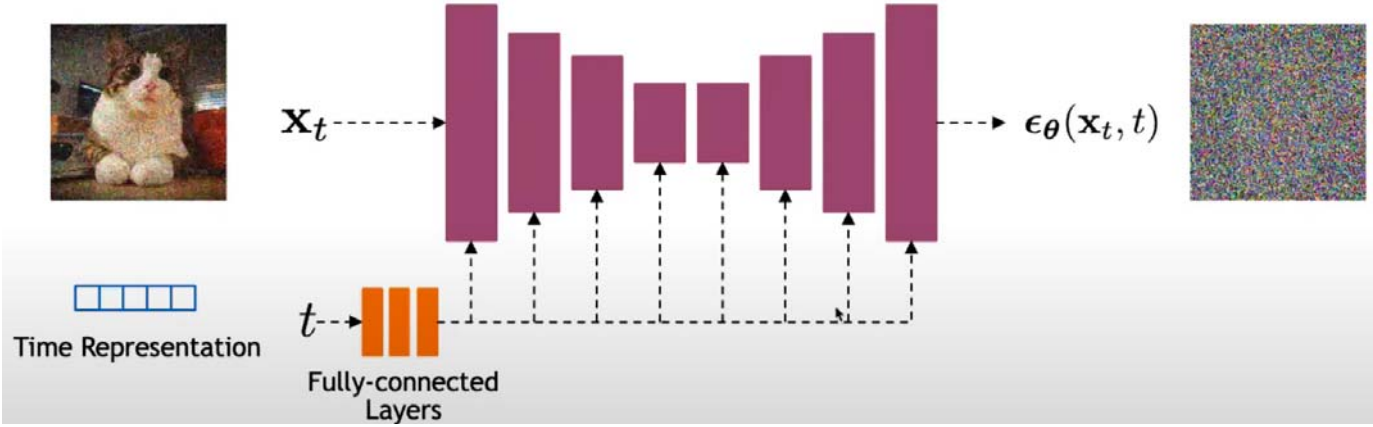
$$p_\theta(\mathbf{x}_{0:T}) = p(\mathbf{x}_T)\prod_{t=1}^{T} p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) \quad p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \boldsymbol{\mu}_\theta(\mathbf{x}_t,t), \boldsymbol{\Sigma}_\theta(\mathbf{x}_t,t))$$



An example of training a diffusion model for modeling a 2D swiss roll data

Diffusion Model

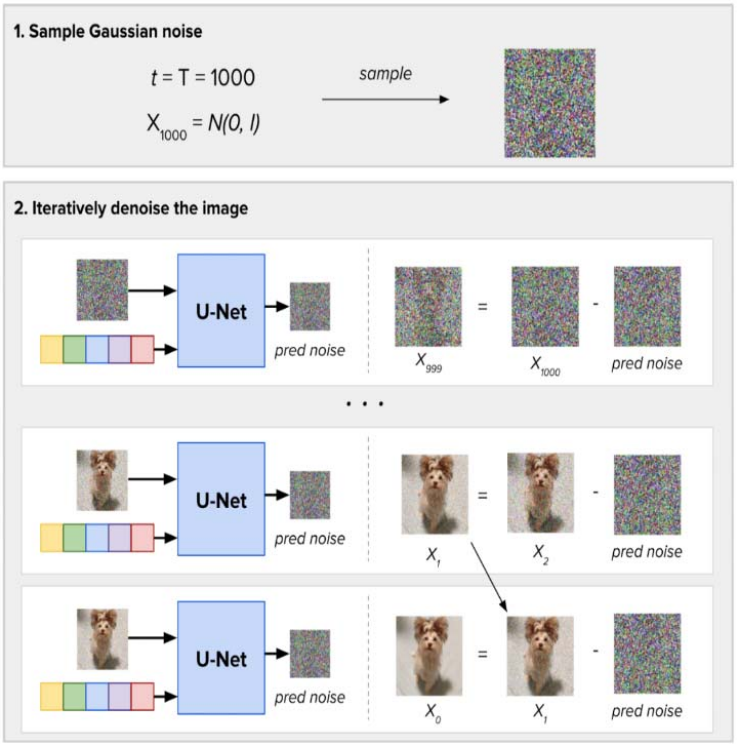
- **Denoising Neural Network Architecture**



$$L_{\text{simple}}(\theta) := \mathbb{E}_{t,\mathbf{x}_0,\boldsymbol{\epsilon}}\left[\, \|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}\theta(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}, t)\|^2 \,\right]$$

$$\begin{aligned} Loss_{Diffusion} &= D_{KL}(q(z \mid x_0) \| P_\theta(x_0 \mid z)) - E_{z\sim q(z|x)}[\log P_\theta(z)] \\ &= D_{KL}(q(z \mid x_0) \| P_\theta(z)) + \sum_{t=2} D_{kL}(q(x_{t-1} \mid x_t, x_0) \| P_\theta(x_{t-1} \mid x_t)) - E_q[\log P_\theta(x_0 \mid x_1)] \end{aligned}$$

Regularization *Denoising Process* *Reconstruction*



Diffusion Model

- **Use Cases**
  - **Image generation via prompting:** generate images based on textual prompts or conditioning inputs, allowing for controlled and customizable image synthesis.
  - **Image super resolution:** enhance the resolution and quality of low-resolution images, generating high-resolution versions with enhanced details and sharpness.
  - **Domain adaptation and style transfer:** transfer the style or characteristics of one image or domain to another, enabling the adaptation of models trained on a source domain to perform well on a target domain with different visual characteristics.
  - **Image inpainting:** fill in missing or corrupted parts of an image, reconstructing the missing details to create visually complete and coherent images.
  - **Image outpainting:** expand the image outside its borders creating continuity and generating a bigger image.

Diffusion Model

- **Well-known Diffusion Models for Image Generation**
  - GLIDE
  - Stable Diffusion
  - DALL-E 2
  - Midjourney
  - Deep Dream Generator
  - Imagen
  - Firefly



High-Resolution Image Synthesis with Latent Diffusion Models



Hierarchical Text-Conditional Image Generation with CLIP Latents



GLIDE: Towards Photorealistic Image Generation and Editing with Text-Guided Diffusion Models

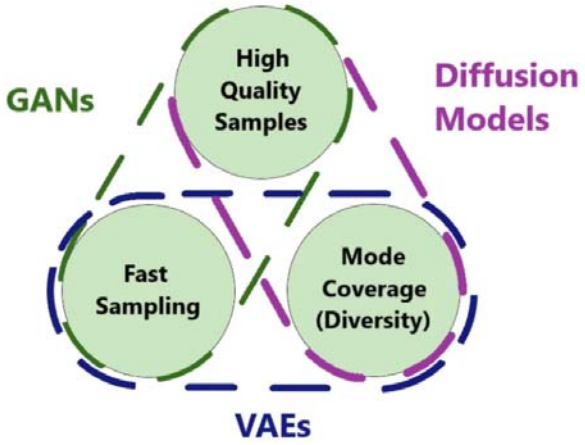


Photorealistic Text-to-Image Diffusion Models with Deep Language Understanding

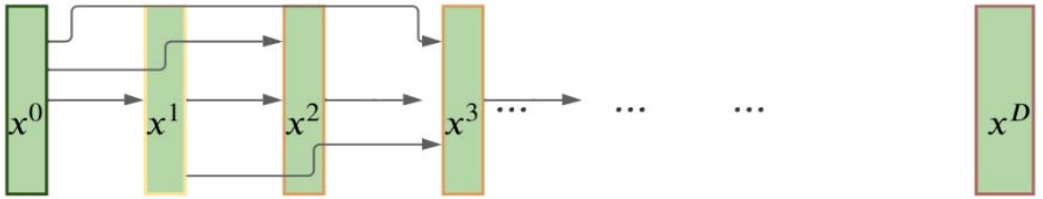
Diffusion Model

- **Key Takeaways**
  - Diffusion models are generative models that simulate how data is made by using a series of invertible operations to change a simple starting distribution into the desired complex distribution.
  - Compared to traditional generative models, diffusion models have **better image quality**, **interpretable latent space**, and **robustness to overfitting**.
  - Diffusion models have **diverse applications** across several domains, such as text-to-video synthesis, image-to-image translation, image search, and reverse image search.
  - Diffusion models excel at generating realistic and coherent content based on **textual prompts and efficiently handling image transformations and retrievals**. Popular models include Stable Diffusion, DALL-E 2, and Imagen.

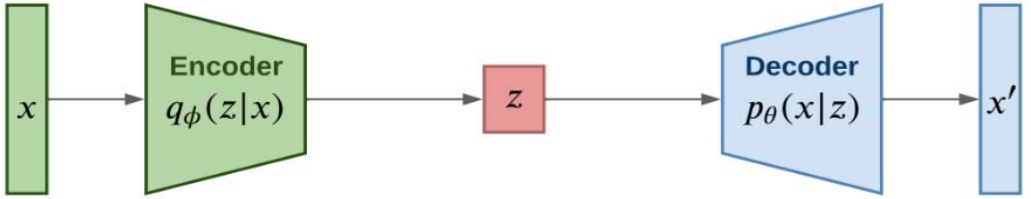
Generative Models



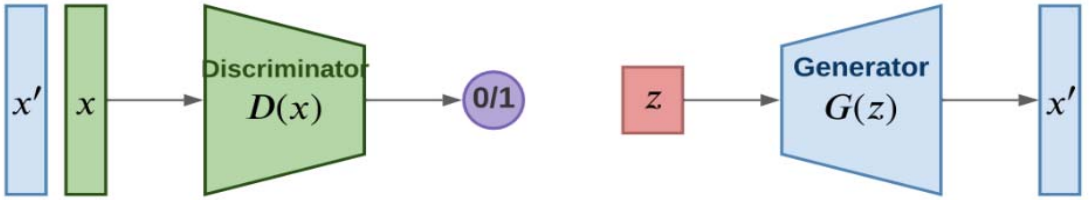
Autoregressive model: Learn conditional of each variable given past



VAE: Maximize variational lower bound



GAN: Adversarial training



Diffusion Model: Gradually add Gaussian noise and then reverse



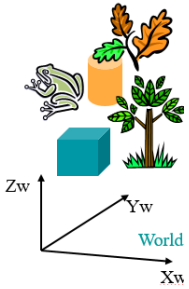
Wrap up

Class overview

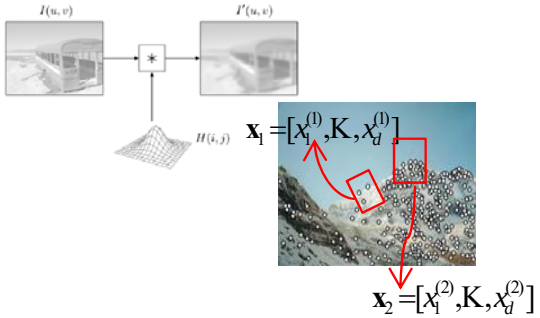
Early Vision — *Middle-level Vision* — *High-level Vision*

Input → **Pixels** → **Image** → **Video** → **Machine Learning**

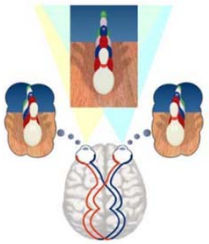
Camera Model
Image Geometry



Linear Filter
Feature Detection
Feature Description



Homography
Stereo Vision



Motion
Tracking



Classification
Detection &
Segmentation

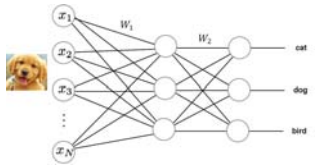


Image
Generative AI



Acknowledgements

- The materials of class are built upon previous efforts of:
  - Ioannis (Yannis) Gkioulekas, Carnegie Mellon Univ., Computer Vision 16-385
  - Noah Snavely and Abe Davis, Cornell Univ., Computer Vision CS5670
  - Fei-Fei Li, Juan Carlos Niebles, and Ranjay Krishna, Stanford Univ., Computer Vision CS131, CS231n
  - Jianbo Shi, Univ. of Pennsylvania, Computer Vision and Computational Photography CIS581
  - Jana Kosecka, George Mason Univ., Computer Vision CS 682
  - Ahmed Elgammal, Rutgers Univ., Computer Vision CS 534
  - Nitin J. Sanket, Univ. of Maryland, Computer Vision CMSC426
  - Per-Erik Forssén, Linköping Univ., Visual Object Recognition
  - Kris Kitani, Bob Collins, Srinivasa Narashiman, Martial Hebert, Alyosha Efros, Ali Faharadi, Deva Ramanan, Yaser Sheikh, Todd Zickler, Steve Seitz, Richard Sezlinsky, Larry Zitnick, Noah Snavely, Lana Lazebnik, Kristen Grauman, Yung-Yu Chuang, Tinne Tuytelaars, Antonio Torralba, Rob Fergus, David Claus, Dan Jurafsky, and Ioannis Gkioulekas

Geometric Camera Models

- Image Transformations
  - Rigid Transformation
  - Similarity Transformation
  - Affine Transformation
  - Perspective Transformation

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} a & b & e \\ c & d & f \\ g & h & i \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

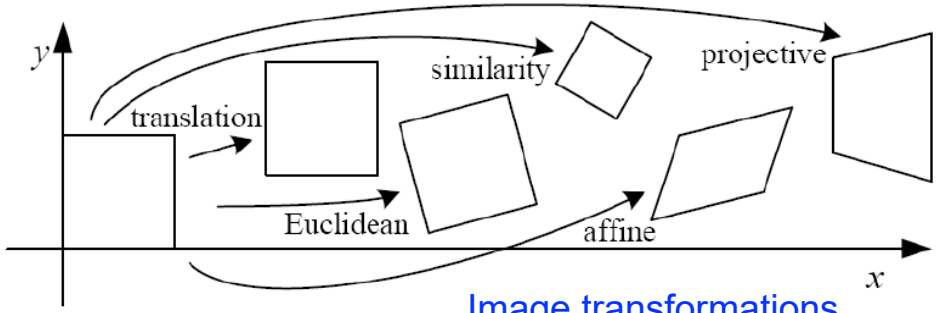
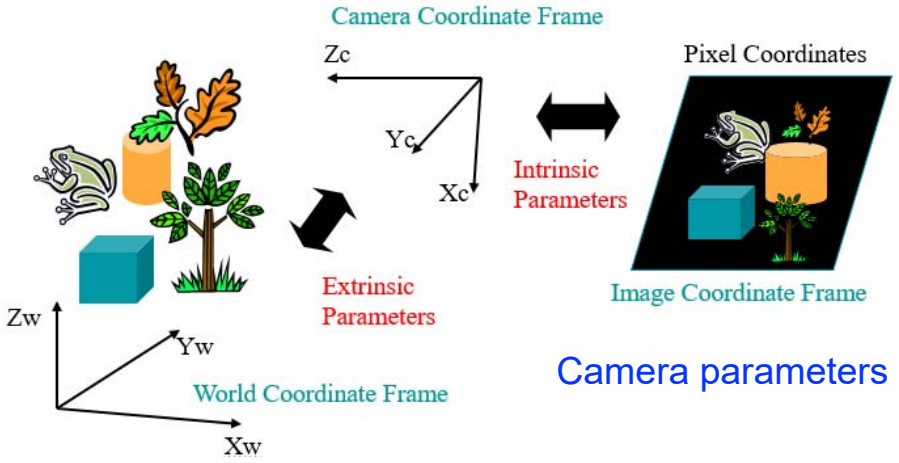


Image transformations

- Geometric Camera Models



Camera parameters

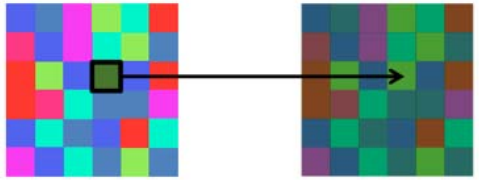
More General Camera Model

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \underbrace{\begin{bmatrix} fs_x & fs_\theta & o_x \\ 0 & fs_y & o_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}}_{\text{Intrinsic parameters}} \underbrace{\begin{bmatrix} R & t \\ 0_{1\times 3} & 1 \end{bmatrix}}_{\text{Extrinsic parameters}} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

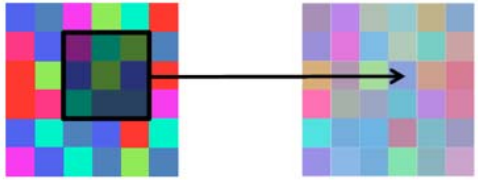
Linear Filters

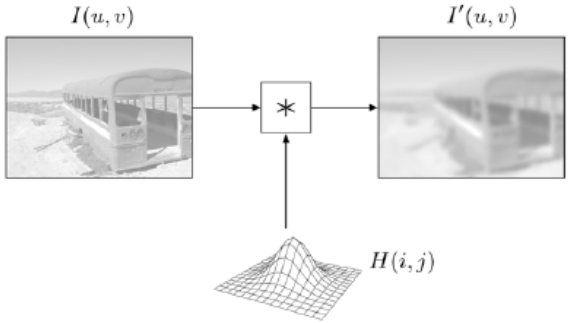
- Filters
  - Linear Filter
  - Gaussian Filter / Derivative of Gaussian Filter
- Edge detecting
  - Sobel Filter

- Point processing

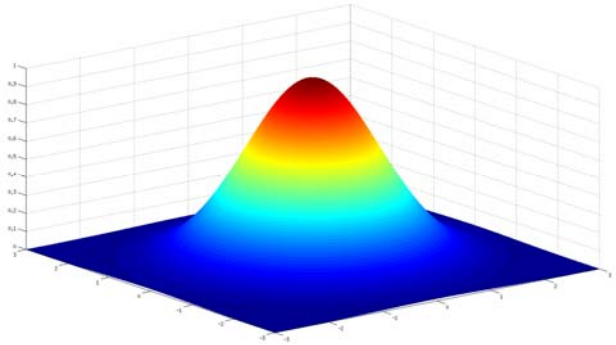


- Filters

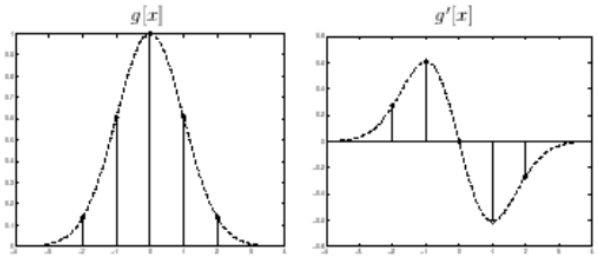




Filter function



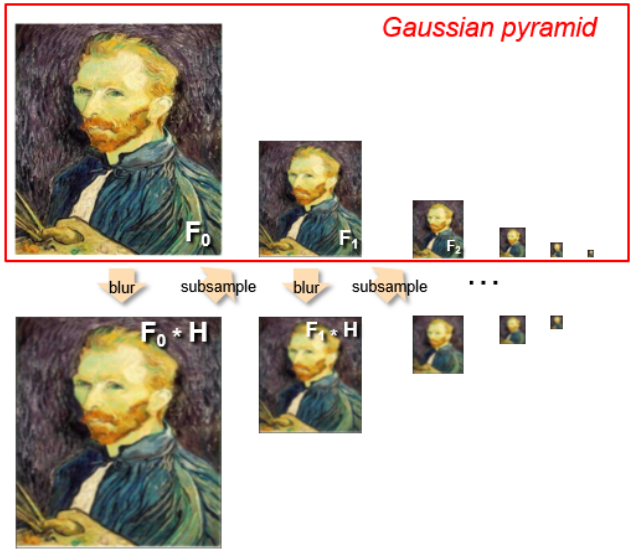
Gaussian Filter



DoG

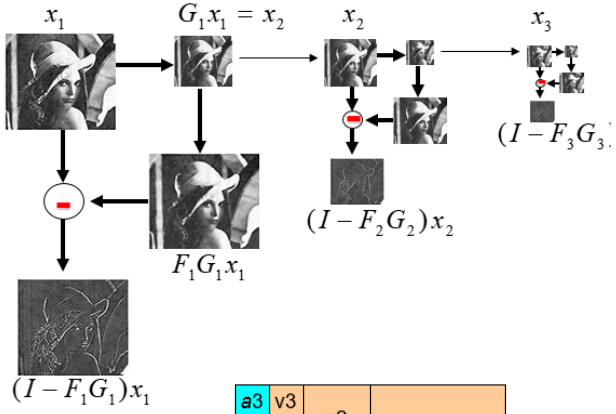
Image Pyramids

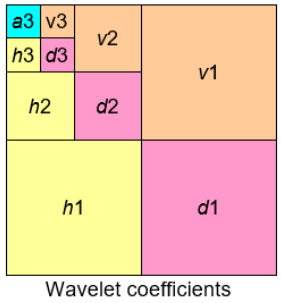
- Gaussian – Laplacian Image pyramid
- Interpolation



Gaussian Pyramid

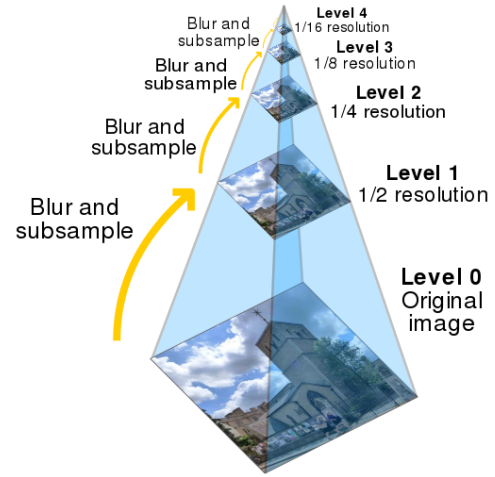
Laplacian Pyramid

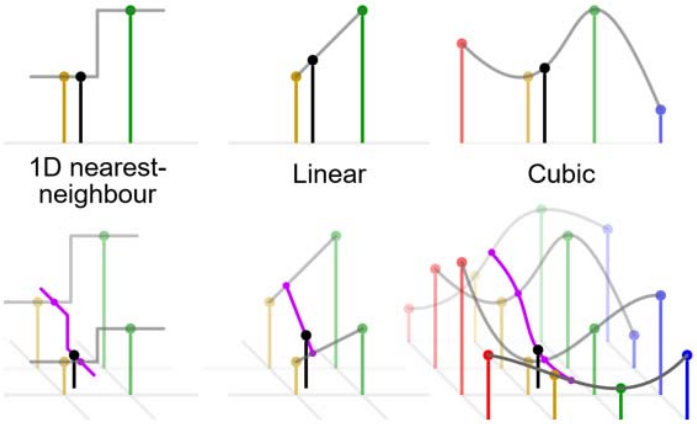




Wavelet coefficients

Wavelet transformation





Interpolation techniques

성균관대학교 SUNG KYUN KWAN UNIVERSITY

Local Features Detection



- Local Feature
- Feature Detectors
  - Edge detection
    - Prewitt, Sobel, Robert's cross operator
  - Line detection
    - Canny edge detector
    - Hough transform
  - Corner detection
    - Harris corner detection
  - Blob detection
    - Laplacian of Gaussian

-1	0	1
-1	0	1
-1	0	1

(Prewitt)

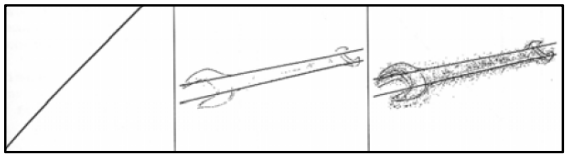
-1	0	1
-2	0	2
-1	0	1

(Sobel)

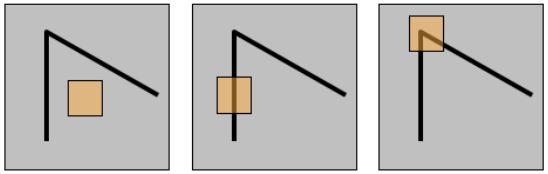
Edge filters

1	0
0	-1

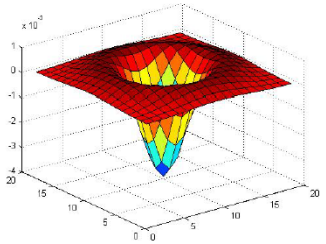
(Robert)

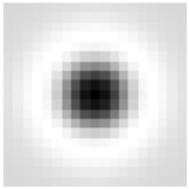


Hough Transform



Harris corner detection

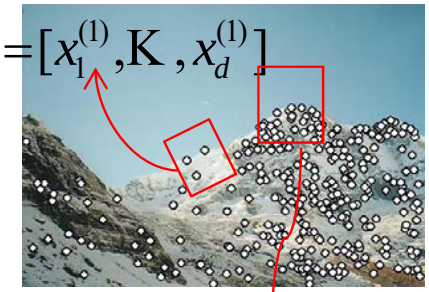


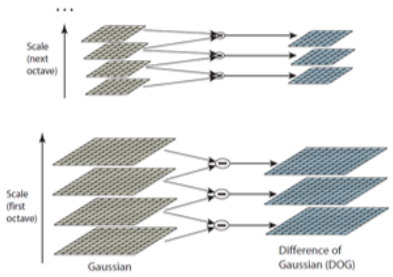


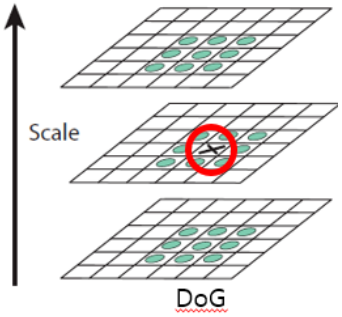
Laplacian

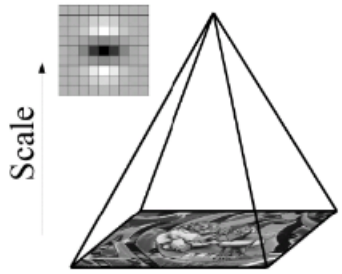
Features Descriptors

- Feature Description
- Feature Descriptors
  - SIFT, SURF, HOG
  - BRIEF, ORB







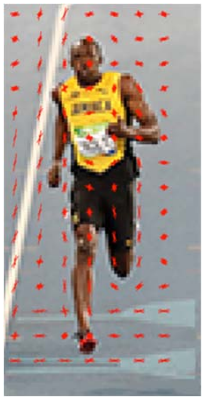


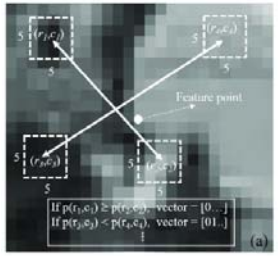
SIFT

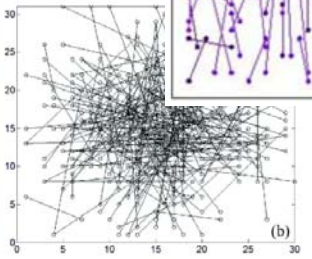


SURF

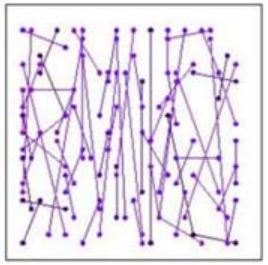
HOG







BRIEF

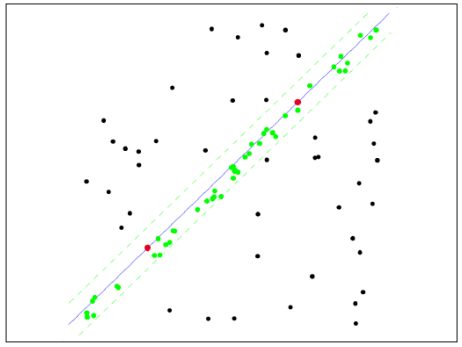


ORB

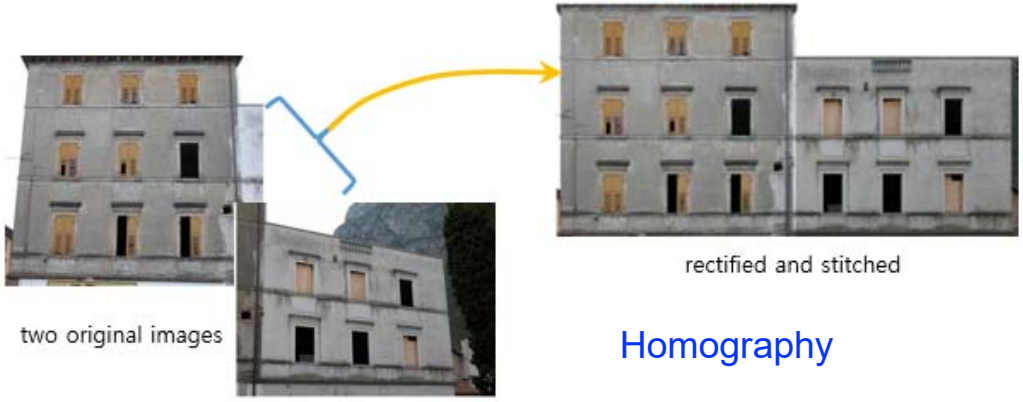
Features Matching



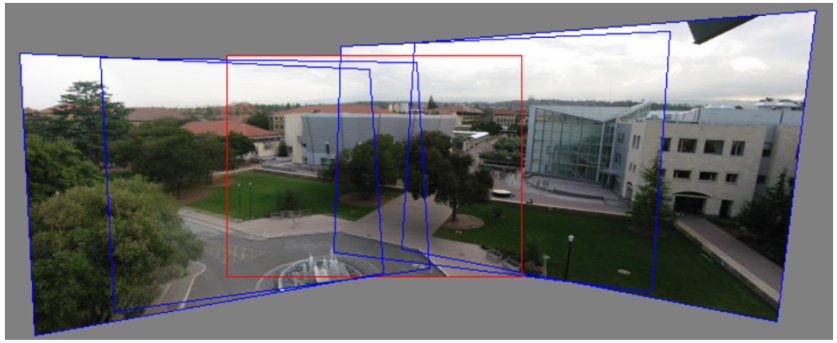
- Feature Matching
  - Fitting, RANSAC
- Image Alignment
  - Homography
  - Rectification
  - Stitching



RANSAC



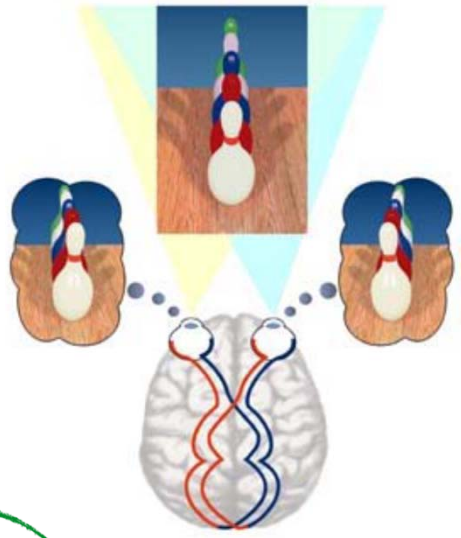
Homography



Stitching

Stereo Vision

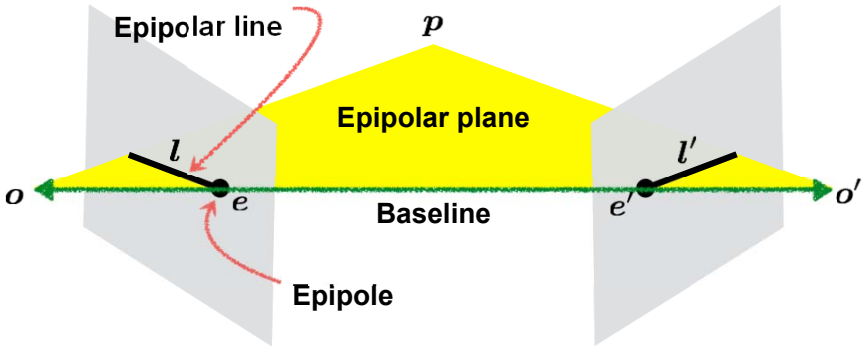
- Two-view Geometry
  - Epipolar Geometry
  - Essential matrix
  - Fundamental matrix
- Stereo Vision
  - Stereo Rectification
  - Stereo Matching
  - Disparity & Depth



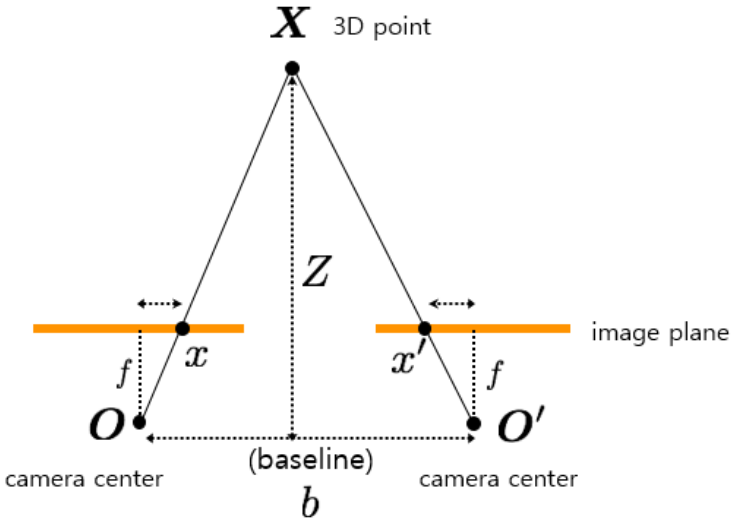


Disparity & Depth





Epipolar Geometry



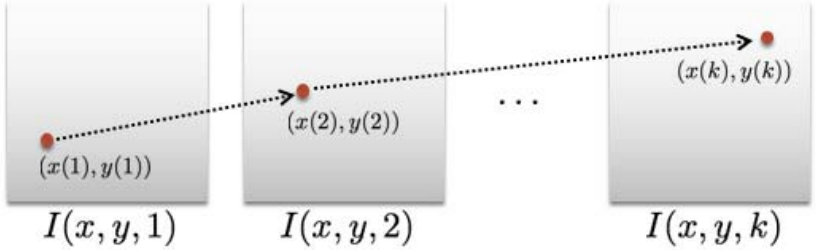
Disparity & Depth

Motion Tracking

- Motion
- Optical Flow
  - Lucas-Kanade
  - Horn-Schunck
- Visual Tracking
  - Kanade-Lucas-Tomasi
  - Mean shift tracking
  - Cam shift tracking

$$I_x u + I_y v + I_t = 0$$

$$\begin{bmatrix} I_x(\boldsymbol{p}_1) & I_y(\boldsymbol{p}_1) \\ I_x(\boldsymbol{p}_2) & I_y(\boldsymbol{p}_2) \\ \vdots & \vdots \\ I_x(\boldsymbol{p}_{25}) & I_y(\boldsymbol{p}_{25}) \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} = - \begin{bmatrix} I_t(\boldsymbol{p}_1) \\ I_t(\boldsymbol{p}_2) \\ \vdots \\ I_t(\boldsymbol{p}_{25}) \end{bmatrix}$$



$$\min_{\boldsymbol{u},\boldsymbol{v}} \sum_{ij} \left\{ E_d(i,j) + \lambda E_s(i,j) \right\}$$

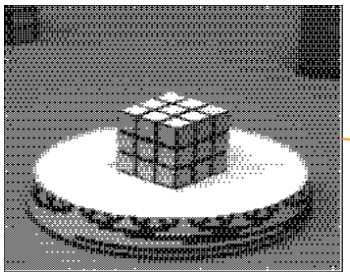


Cam shift tracking

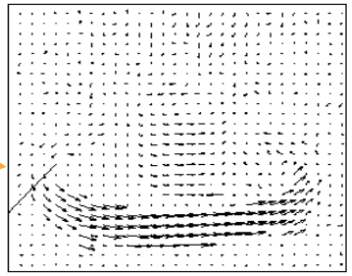


Cam shift tracking

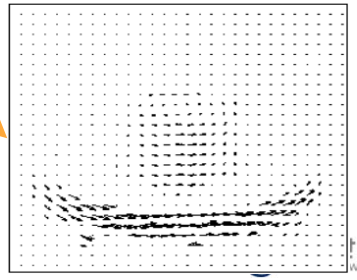




Lucas-Kanade

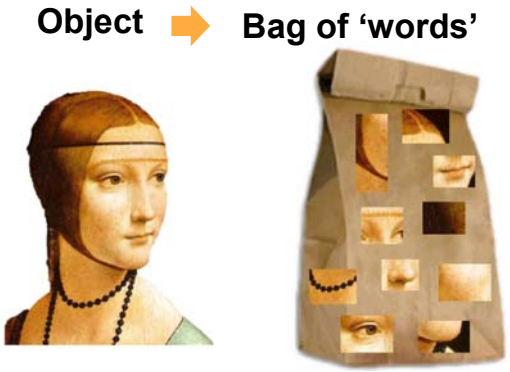


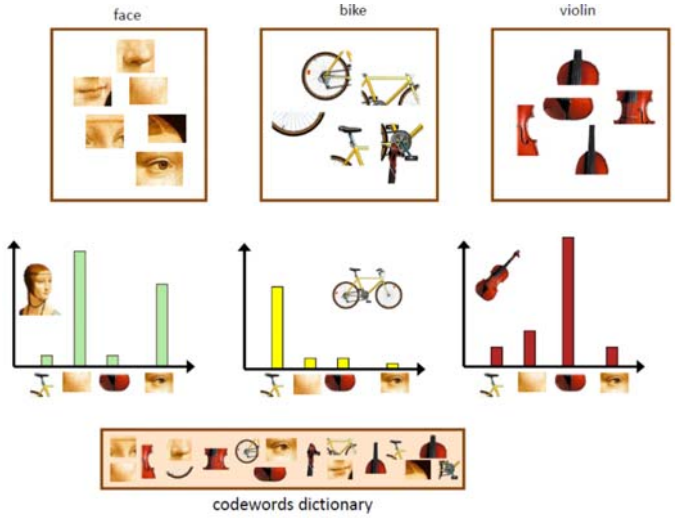
Horn-Schunck



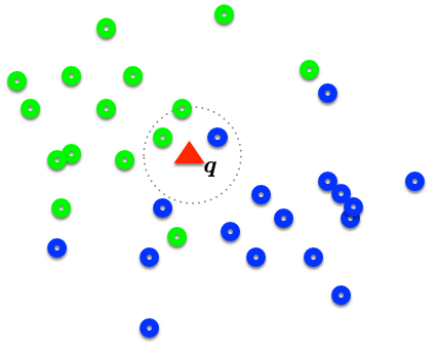
Classification

- Bag of words

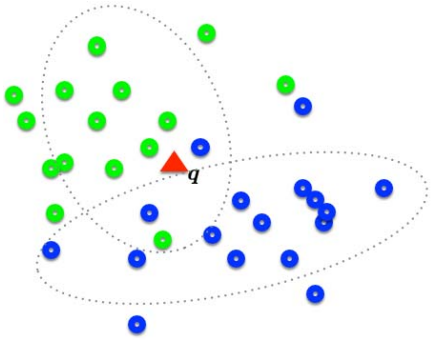




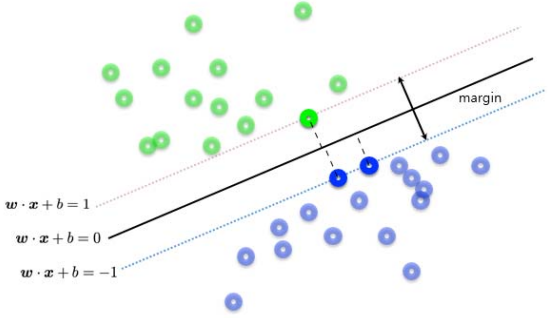
- Classifier
  - K-Nearest Neighbor
  - Naïve Bayes
  - Support Vector Machine



K-Nearest Neighbor



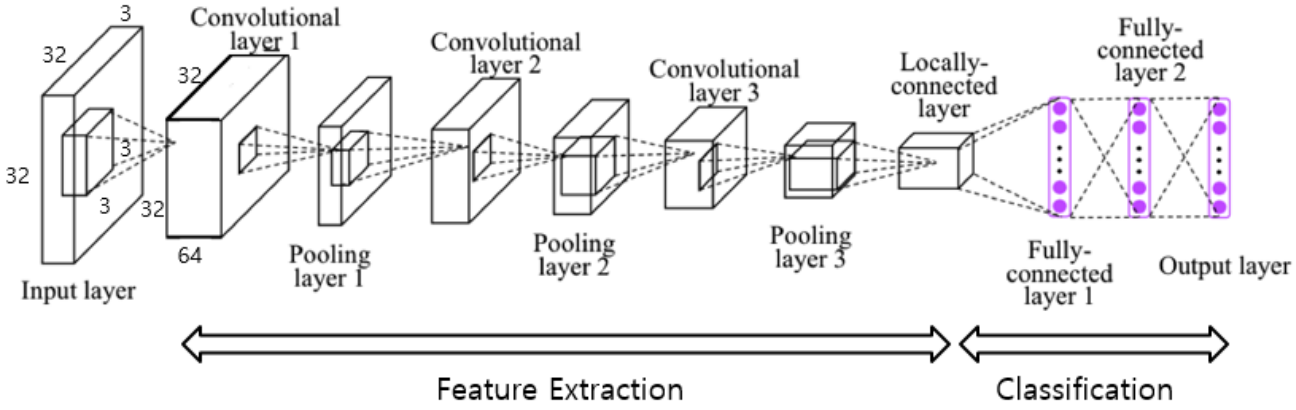
Naïve Bayes

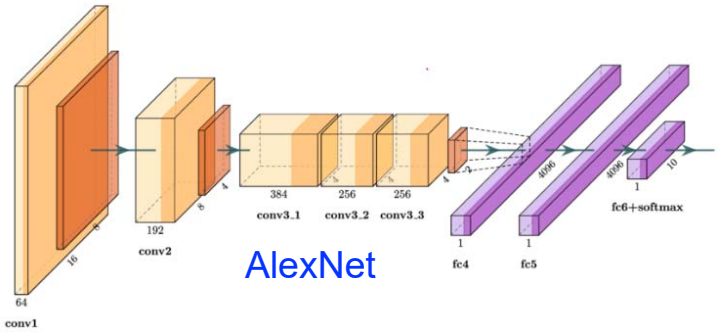


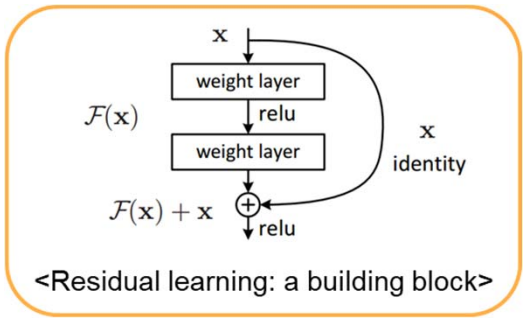
Support Vector Machine

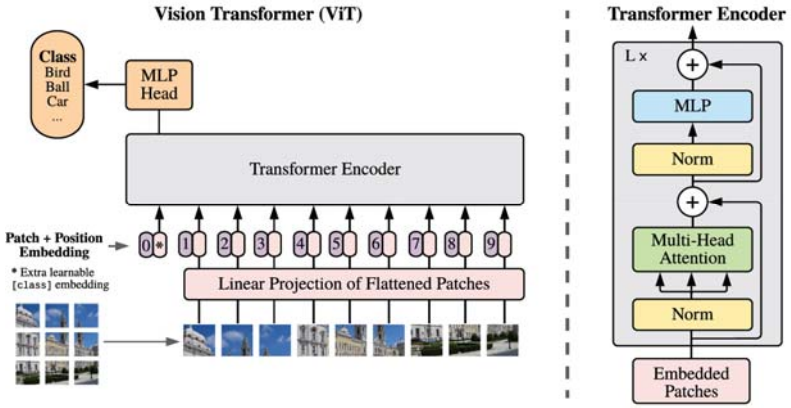
Classification (2)

- Image Classification
- ConvNet (CNN)
- AlexNet / ResNet
- Vision Transformer









Detection & Segmentation

- Detection & Segmentation
- Object Detection
  - RCNN (Fast/Faster)
  - SSD / YOLO
- Segmentation
  - FCN (for semantic)
  - Mask-RCNN (for instance)
- Data Set / Evaluation



CIFAR-10 (Canadian Institute for Advanced Research, 10 classes)
The CIFAR-10 dataset (Canadian Institute for Advanced Research, 10 classes) is a subset of the Tiny Images dataset and consists of 60000 32x32 color images. The images are labelle...
14,173 PAPERS • 98 BENCHMARKS

ImageNet
The ImageNet dataset contains 14,197,122 annotated images according to the WordNet hierarchy. Since 2010 the dataset is used in the ImageNet Large Scale Visual Recognition...
13,519 PAPERS • 41 BENCHMARKS



MS COCO (Microsoft Common Objects in Context)
The MS COCO (Microsoft Common Objects in Context) dataset is a large-scale object detection, segmentation, key-point detection, and captioning dataset. The dataset consists of...
10,220 PAPERS • 93 BENCHMARKS



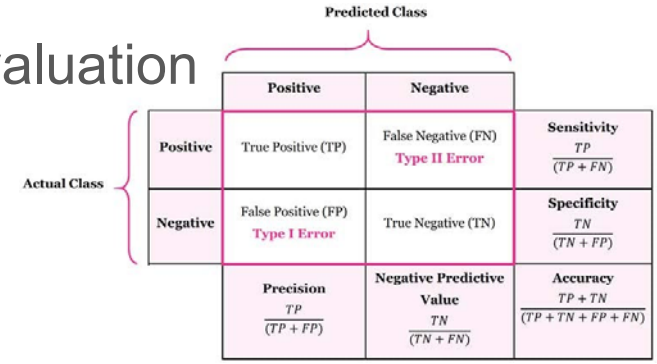
CIFAR-100
The CIFAR-100 dataset (Canadian Institute for Advanced Research, 100 classes) is a subset of the Tiny Images dataset and consists of 60000 32x32 color images. The 100 classes in...
7,706 PAPERS • 52 BENCHMARKS

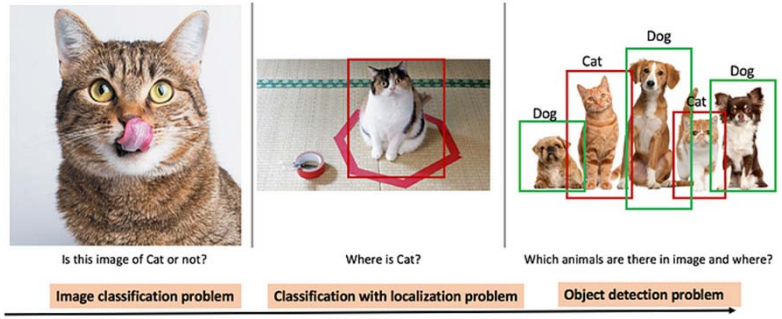


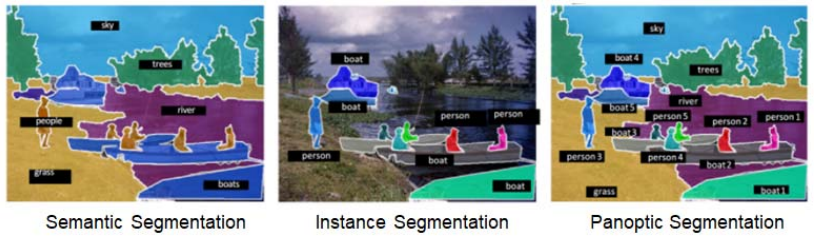
MNIST
The MNIST database (Modified National Institute of Standards and Technology database) is a large collection of handwritten digits. It has a training set of 60,000 examples, and a test...
7,004 PAPERS • 52 BENCHMARKS

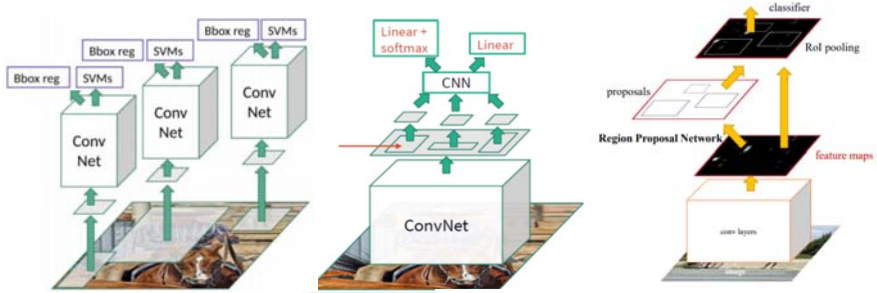


Cityscapes
Cityscapes is a large-scale database which focuses on semantic understanding of urban street scenes. It provides semantic, instance-wise, and dense pixel annotations for 30 class...
3,334 PAPERS • 54 BENCHMARKS









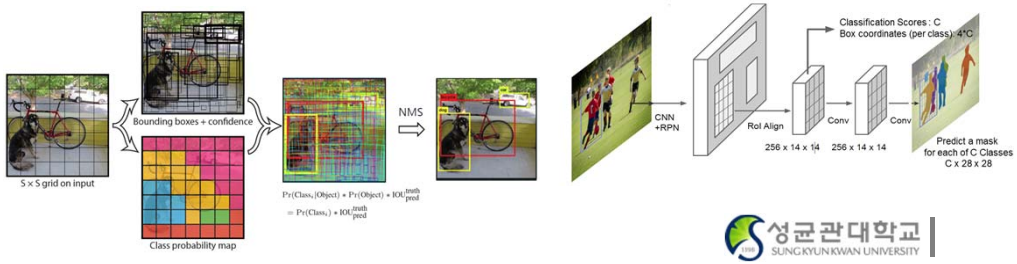
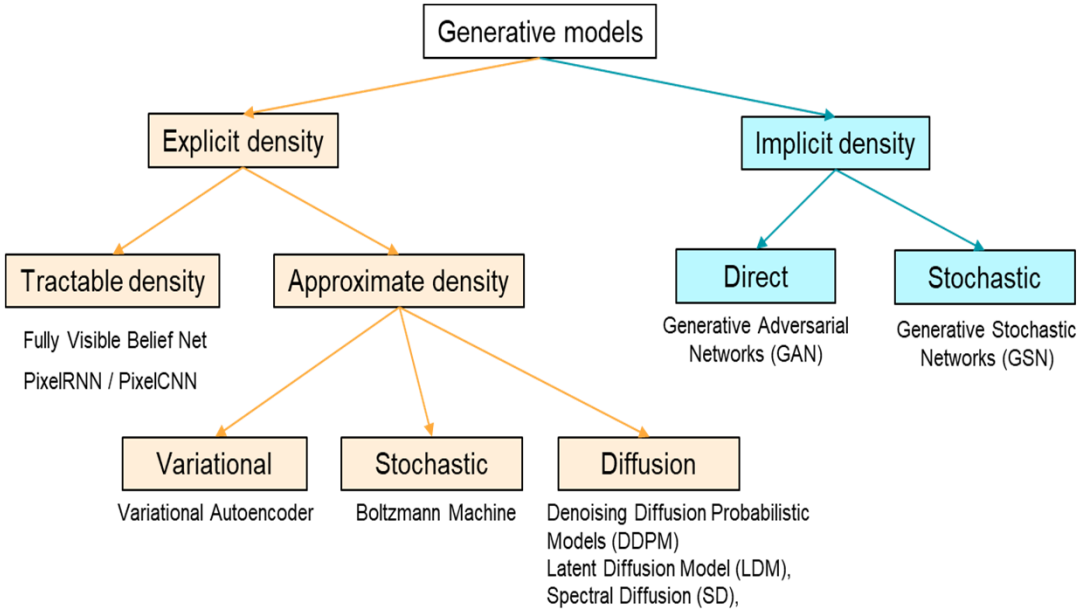
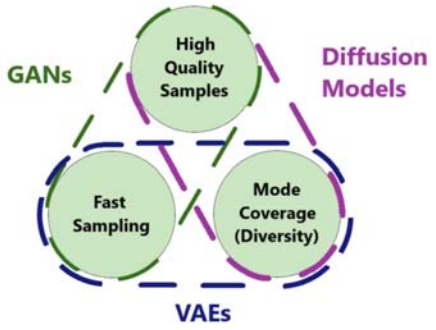
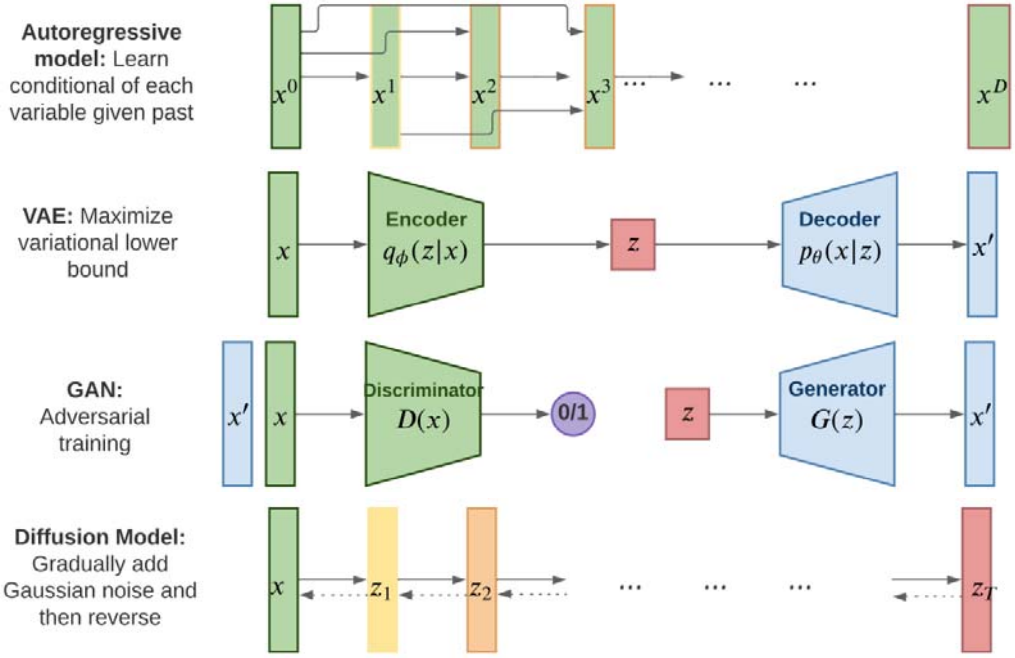


Image Generative Model

- Given training data,
  generate new samples from same distribution







성균관대학교 SUNG KYUN KWAN UNIVERSITY

Thank you for the semester

